

some shift operation. If table T can only be reached after a reduce operation, and the next token is an error token, the error will be detected before the reduce operation is begun, through some X entry in the f table.

We therefore keep every X entry in the T_0 table. We then follow every shift entry in the action table through its goto table; if a shift action ends in a transfer to table T , then we must keep every X entry in the action part of table T .

Note that the action part of a table T will carry X entries or “@” entries; no mixture of the two is possible.

Example. Consider the tables for G_0 given previously. All the X entries in the goto table may be changed to “@”. All the X entries in the action table T_0 must be retained. However, all the X entries in the action tables 1, 2, 3, 8, 9, 10, 13, 14, 18, 19, and 20 may be changed to “@”, since none of these tables is the target of a shift action. For example, table 1 appears in only one place in g , as $g(0, E)$; it is not the target of a shift. Similarly, table 2 only appears in $g(0, T)$. On the other hand, table 5 appears as $g(0, “(”)$, and $f(0, “(”) = \text{shift}$, so table 5 must contain X entries.

Postponement of Error Checking

Consider tables 15 and 21 for G_0 :

action, f							goto, g							
	a	+	*	(	)	ϵ	E	T	F	a	+	*	(	)
15	X	5	5	X	X	5	X	X	X	X	X	X	X	X
21	X	5	5	X	5	X	X	X	X	X	X	X	X	X

These two tables are clearly equivalent, except for the “)” and “ ϵ ” columns in the action part. Suppose we changed these into the following tables:

action, f							goto, g							
	a	+	*	(	)	ϵ	E	T	F	a	+	*	(	)
15	X	5	5	X	5	5	X	X	X	X	X	X	X	X
21	X	5	5	X	5	5	X	X	X	X	X	X	X	X

We have changed the parser so that in certain circumstances, some reduce operations will be permitted that otherwise would be considered an error. Eventually, however, the error must be detected, and must not lead to an “@” in an f -table. We must therefore examine the consequences of each such change of X to *reduce* n .

Let us therefore consider a set of tables S_T that are equivalent except for some X entries in the action part that we would like to change into reduce operations. We need an algorithm that will test whether we can safely do this, and that will indicate other necessary changes (e.g. “@” entries that must become X). Note first that X and “@” entries in the set S_T may be considered the same; we can always change “@” entries back to X as needed. In what follows, assume that every such change is made; “@”s are changed to X’s as needed, and some X’s in f are changed to reduces. (We must later retract all this, if the test for its permissibility is negative.)

Consider one such table entry in table T, with an entry $f(T, h) = X$ that we would like to change to *reduce n*, where production n is $A \rightarrow w$. In the reduction, $|w|$ table names are dropped from the stack, exposing some table T' . We must next consider $g(T', h)$. If this is “@”, we can change the “@” to X, to catch the error there. However, $g(T', h)$ may be some table T'' . We must then examine $f(T'', h)$; this must either be *error* or another reduce, e.g., *reduce m*. If the latter, the search must continue deeper. If the search leads to some table T_3 such that $f(T_3, h) = \text{shift}$, then the change of $f(T, h)$ from X to *reduce n* cannot be made.

Further development of this reduction algorithm will lead to a difficult combinatorial problem, the optimal selection of subsets of tables that can be made equivalent under this transformation. Suppose that we find that some subset S' is equivalent under error postponement. Its equivalence is in general the result of some modifications elsewhere in the table (some changes of “@” to X), and they must be preserved upon the next equivalence consideration. The problem is that one set of tables S' may not be equivalent unless some other set S'' is similarly shown to be equivalent, or a set S' cannot be equivalent if S'' is made equivalent. We see that in general, an optimal reduction can only be made if all possible combinations are tested with the objective of finding the least final table set.

A nearly optimal approach is the following:

Step 1. Identify those subsets of tables that are candidates for an equivalence reduction. These must necessarily be disjoint. Order the subsets in order of decreasing size, and apply step 2 to them in that order.

Step 2. Given a subset, make the necessary changes in its tables for an error postponement equivalence and test the validity of the transformation. Make whatever other changes are necessary. Each of these changes must be revocable. If the test is positive, fix the changes, merge the states, and apply step 2 to the next state subset. If the test is negative, revoke all the changes and apply step 2 to the next state subset.

In this algorithm, we have no assurance that an optimal solution will be found. It is conceivable that a merger of a set of three states will preclude

mergers of several subsequent sets of two states each.

The NEXT Function

A critical step in this algorithm is the identification of that set of tables that can appear just below some handle in the stack. Let this set be designated $NEXT(T, i)$, where T is a top-of-stack table and i is a production in $f(T, h)$ for some token h . Let production i be $A \rightarrow w$, where $w = w_1 w_2 \dots w_n$. Then table T' will be in $NEXT(T, i)$ if and only if a goto sequence of the following kind exists:

$$\begin{aligned} g(T', w_1) &= T_1; \\ g(T_1, w_2) &= T_2; \\ &\vdots \\ g(T_n, w_n) &= T. \end{aligned}$$

This sequence follows from the property of an LR parser that its finite-state machine (represented by the goto tables) will accept the viable prefix of a right-most sentential form, and the correspondence of the tables (or states) on the stack to the terminal and nonterminal symbols of the viable prefix.

Now the error postponement algorithm can be given in more detail:

1. Identify a set of tables P that are candidates for error postponement. These tables will agree, except for conflicts between “@”, X , and *reduce n* .
2. Among the tables P , change each @ to X and each X to *reduce n* as needed in order that they all agree. Let production n be $A \rightarrow w$. Then there must not exist a path in the goto table from T_0 to T on a suffix of w ; all paths from the initial table must either spell w fully, or spell some string w' whose suffix is w . This condition guarantees that the stack will contain enough states to permit a reduction. For every change of X to a *reduce n* , perform step 3.
3. Given a change of some X to a *reduce n* in $f(T, h)$ (table T , token h), then repeat step 4 for every table T' in $NEXT(T, n)$, where n is the production number in step 2 above.
4. Given a table T' and token h from step 3. The value of $g(T', h)$ may be a table T'' , “@”, or X . If “@”, change it to X to catch the error. $f(T', h)$ will be either “@”, X , *shift*, or *reduce m* . If “@”, change “@” to X . If *shift*, the set P cannot be merged; halt. If *reduce m* , table T' may be in set P and $m = n$; if so, ignore it, otherwise set P cannot be merged; halt.

If the algorithm halts agreeably in step 4, then the set P can be merged as indicated in step 2.

Example. Consider tables 15 and 21 in the canonical parser for G_0 . We are interested in changing $f(15, "(")$ to 5 and $f(21, "\epsilon")$ to 5. Production 5 is $F \rightarrow (E)$, and an examination of the g table shows that tables 0, 6, and 7 will lead to $f(15, "(")$ on the handle (E) ; $\text{NEXT}(15, 5) = \{0, 6, 7\}$. In every case, $f(T', "(") = X$, where T' is in $\text{NEXT}(15, 5)$, so this appears safe. (No "@" entry in g need be changed, incidentally).

Next consider changing $f(21, \epsilon)$ to 5. (The ϵ means that the parser has reached the end of the input list). Now $\text{NEXT}(21, 5) = \{5, 12, 16, 17\}$ from the g table. Again, in every case, $f(T', \epsilon) = X$, and no "@" entry in a g table need be changed. We therefore conclude that tables 15 and 21 may be safely merged through error postponement.

Removal of Single Production Transitions

Consider the single production $E \rightarrow T$ in grammar G_0 , and a derivation

$$E \Rightarrow T \Rightarrow T * F$$

for example. Since the reduce step associated with $E \Rightarrow T$ rarely (if ever) is needed as a semantics action step, we would like to transform the canonical parsing tables such that the derivation is effectively

$$E \Rightarrow T * F$$

The result is fewer parser operations and a smaller table.

Typical programming language grammars contain many such productions. They generally appear in the definition of the expression operators, and there are as many single productions as there are precedence hierarchy levels in the language. A parenthesized expression often calls for a long sequence of single production reduction steps. We would clearly like to be able to remove as many single production reduction steps as possible.

Suppose, then, that the action corresponding to a production $A \rightarrow B$ is to be eliminated from the canonical tables; we want no reduce step on this production to ever appear. Any pair of steps of the form

$$xAy \Rightarrow xBy \Rightarrow xwy$$

where $B \rightarrow w$ is some production is to be effectively replaced by

$$xAy \Rightarrow xwy$$

during parsing.

We examine every table T' containing a *reduce* p action, where production

p is $A \rightarrow B$. Let T'' be a table in $\text{NEXT}(T', p)$. Then table T'' can be removed only if, for all tokens h :

- { 1. $f(T', h) = f(T'', h)$
- or 2. One of $f(T', h)$, $f(T'', h)$ is $@$
- or 3. $f(T', h) = \text{reduce } p$ }

and

- { 4. $g(T', h) = g(T'', h)$
- or 5. One of $g(T', h)$, $g(T'', h)$ is $@$ }

If this condition holds, then we can eliminate table T'' by setting $f(T', h) := f(T'', h)$ if $f(T', h) = @$, and by setting $g(T', h) := g(T'', h)$ if $g(T', h) = @$. Then every reference to T'' can be changed into a reference to T' , throughout the table set.

Note: (1) When T'' is removed from the table, NEXT will be affected in general. It must be recomputed for each such reduction. (2) The algorithm will fail if some nonterminal derives only the empty string. (3) The algorithm will fail if some single production $A \rightarrow C$, where C is not B , also exists.

Condition (2) is rather stringent; useful programming languages often contain one or more nonterminals that only derive the empty string. (They are useful as place markers for certain semantic steps). Condition (3) is less likely.

The removal of these restrictions requires a more elaborate algorithm. Soisalon-Soininen [1977] gives a bibliography of work on this problem and a general solution.

Equivalence

Consider a set of canonical tables, possibly containing $@$ entries, which may have been subjected to one or more of the above reductions. We may then perform an equivalence reduction along the lines described in chapter 3. Let the set of canonical tables be partitioned into their k -equivalence classes (k is an arbitrary integer, not the " k " of $\text{LR}(k)$). We then refine the partition to yield a $(k+1)$ -equivalence partition as follows:

Let tables T and T' belong to one of the k -equivalence sets P . Then T and T' belong to the same $(k+1)$ -equivalence set if, for every token h :

$$\{ f(T, h) = f(T', h), \text{ or one of } f(T, h), f(T', h) = @ \}$$

and

$$\{ g(T, h) \text{ and } g(T', h) \text{ are both in } P, \text{ or one of } g(T, h), g(T', h) = @ \}$$

Refinement should be repeated until no proper refinement is obtained. Then each partition represents an equivalence class of tables. Each such class may be merged as follows:

Let $T = T'$, i.e., both belong to the same equivalence class. We eliminate T' , retaining T , as follows:

1. If $f(T, h) = @$ for some token h , then set $f(T, h) := f(T', h)$. Note that $f(T', h)$ may also be $@$.
2. If $g(T, h) = @$ for some token h , then set $g(T, h) := g(T', h)$.
3. Replace T' by T everywhere in the canonical tables.
4. Delete table T' .

The initial partition might as well be the single set consisting of the complete set of tables. The first pass through the equivalence reduction will usually yield many 1-equivalent sets, and a typical grammar will require no more than two or three passes.

Exercises

1. Construct a set of LR(1) canonical tables for the grammar

$G \rightarrow S$
 $S \rightarrow b D$
 $S \rightarrow E e$
 $D \rightarrow D d$
 $D \rightarrow \epsilon$
 $E \rightarrow E s$
 $E \rightarrow s$
 $E \rightarrow S$

Identify the inaccessible table entries, identify states that can be merged through error postponement, states that can be merged by removing the parsing actions associated with the two single productions, and finally identify the equivalent state sets.

2. Develop an algorithm that accepts an LR(1) canonical table set and yields READ, LOOKAHEAD, APPLY, and PUSH tables, with "else" default columns in the LOOKAHEAD and APPLY tables.
3. Write a Pascal program that constructs a set of LR(1) canonical tables. For practical grammars, a large number of LR(1) tables will be generated. Develop efficient means of storing and manipulating the state sets through a mass storage medium (disk or drum).

6.2.4. LALR(1) Tables

A set of LALR(1) tables may be constructed from the set of LR(1) tables as follows:

Recall that an LR(k) item has the form

$$[A \rightarrow x.y, w]$$

where $A \rightarrow xy$ is a production and w is a lookahead string, consisting of terminal tokens. The portion $A \rightarrow x.y$ of the item is called the *core* of the item. Now two items are said to belong to the same LALR(k) parser state if they have the same core. We essentially ignore the lookahead string w in forming a state. In doing so, we arrive at the same state set as in an LR(0) construction, but the items will carry lookahead set information derived from the LR(k) construction process. The lookahead sets may then be used to resolve any inadequate states.

For example, consider the LR(1) construction for grammar G_0 given previously. Its states may be merged as follows to yield a set of LALR(1) states:

(0), (1), (2, 9), (3, 10), (4, 11), (5, 12), (6, 16), (7, 17), (8, 18), (13, 19), (14, 20), (15, 21).

The resulting FSA is isomorphic to the LR(0) machine given in section 6.1. It has two inadequate states (ignoring the lookaheads), the merged states (2,9) and (13,19). However, these are easily resolved through the LR(1) lookahead strings carried into the merger. Thus the merged state (2,9) has the following set of items:

(2,9)	> [E → T., ϵ +)]	reduce (2)
	> [T → T.*F, ϵ + *]	7

The second item clearly applies on a lookahead set $\{*\}$, since token “*” follows the mark. The first item applies on a lookahead set $\{\epsilon, +,)\}$. Since the two lookahead sets are disjoint, the state is resolvable.

An LALR(k) resolution is more powerful in general than either an SLR(k) or an SLALR(k) resolution; i.e., there exist grammars that are resolvable by LALR(k), but not SLALR(k) or SLR(k), and there exists no grammar resolvable by either SLALR(k) or SLR(k) and not LALR(k).

As a practical matter, the construction of an LR(1) item set requires a very large memory, compared to an LR(0) item set. Most practical LALR(1) constructors therefore merge core-items as they are generated. Unfortunately, the lookaheads are also merged and the construction is thereby weakened. Pager [1977] discusses this problem in depth and gives a general merging algorithm that preserves as much lookahead information as possible,

without demanding as much storage capacity as an LR(1) constructor would. In Pager's algorithm, the question of state inadequacy is examined when the merger of a pair of states is being considered. The merger is allowed if an inadequacy is not thereby created and disallowed otherwise. Now such a state merger in general requires further work in that state and in all other states that have been derived from it; Pager's algorithm deals with the problem of fixing up the items that must change in response to the merger. The method is complicated and the reader is referred to his paper for further details.

6.3. Augmented Grammars

An alternative way of generating an LR(0) finite-state machine is through an *augmented grammar*, derived from a context-free grammar G , by the following algorithm:

1. Suppose there are n productions in the grammar. We then create a new set of symbols not already in the grammar, $\#i$, where $1 \leq i \leq n$. We thus have a unique symbol for each production.
2. Create a set of new symbols that correspond to the nonterminals in G , i.e., if A is a nonterminal in G , then create a new symbol A' .
3. Construct a new regular grammar G' from G , as follows:
 - (a) For every production $A \rightarrow w$ in G , with index i , add $A' \rightarrow w \#i$ to grammar G' . (A' corresponds to A).
 - (b) For every nonterminal B in a right-part of a production $A \rightarrow xBy$, add production $A' \rightarrow x B' y$ to grammar G' . (A' corresponds to A , and B' to B .)
4. The nonterminals of G' are the symbols $A', B', \dots$ that correspond to the nonterminals $A, B, \dots$ of G .
5. The terminals of G' are all the remaining symbols—the unmarked nonterminals of G , the terminals of G , and the special symbols $\#i$.
6. Augment G with a special start symbol S'' and a production $S'' \rightarrow S'$.

We now clearly have a regular grammar. Each of the productions are of the form

$$\begin{aligned} A &\rightarrow x && \text{where } x \text{ is a terminal string, or} \\ A &\rightarrow yB && \text{where } y \text{ is a terminal string or empty, and } B \text{ is} \\ &&& \text{a nonterminal symbol.} \end{aligned}$$

A finite-state machine can clearly be constructed for this grammar, by

methods described in chapter 3. Furthermore, any FSA can be transformed into a deterministic FSA. What properties does the deterministic FSA have? Some reflection will reveal that:

1. The states will correspond to subsets of the nonterminals in G' .
2. The start state will correspond to state S'' .
3. The halt states will be entered upon scanning one of the special $\#i$ symbols, since only these productions in the regular grammar end in a terminal symbol.
4. Given some sentence in the language $L(G')$, the machine will of course accept the sentence by falling into a halt state upon scanning the sentence.
5. Now the sentences of $L(G')$ are viable prefixes of sentential forms in G . (We shall prove this shortly.) Therefore our FSA appears to be exactly the LR machine that we wish—it will scan a right-most sentential form uxv , with x the handle, and will find a $\#i$ transition precisely upon scanning the viable prefix ux ; the $\#i$ will indicate the correct production to apply. However, there is no guarantee that $\#i$ will be the only transition from the state reached upon scanning ux . If there is another transition, then we contend the grammar G is not $LR(0)$; the grammar G is $LR(0)$ if and only if for every state S with an out-going “ $\#$ ” transition, there are no other out-going transitions.

Example Consider grammar G_0 . The augmented grammar G_0' is given below:

$$\begin{aligned}
 S'' &\rightarrow E' \\
 E' &\rightarrow E + T \#1 \\
 E' &\rightarrow E + T' \\
 E' &\rightarrow E' \quad \{\text{trivial productions may be dropped}\} \\
 E' &\rightarrow T \#2 \\
 E' &\rightarrow T' \\
 T' &\rightarrow T + F \#3 \\
 T' &\rightarrow T + F' \\
 T' &\rightarrow F \#4 \\
 T' &\rightarrow F' \\
 F' &\rightarrow (E) \#5 \\
 F' &\rightarrow (E' \\
 F' &\rightarrow a \#6
 \end{aligned}$$

The FSA given in figure 6.1 (aside from the lookahead states) may be derived from this regular grammar (left as an exercise).

Let us now show that the viable prefix of a sentential form (augmented by a #i symbol) is derivable in G' . As an example, consider the right-most sentential form

$$E + (E) * a$$

in G_0 . The viable prefix “ $E + (E) \# 5$ ” is derivable in G' as follows:

$$S'' \Rightarrow E' \Rightarrow E + T' \Rightarrow E + F' \Rightarrow E + (E) \# 5$$

The FSA based on G' will move through a sequence of states associated with the string “ $E + (E)$ ”, and the next state will then carry a transition on symbol #5. In figure 6.1, the state sequence is 8, 9, 12, 11, 14, 6. State 6 is associated with production number 5, $F \rightarrow (E)$, and will therefore carry an out-going transition on symbol #5 in the FSA derived from G' .

We may compare the derivation of $E + (E) * a$ in G with the G' derivation given above:

$$E \Rightarrow E + T \Rightarrow E + T * F \Rightarrow E + T * a \Rightarrow E + F * a \Rightarrow E + (E) * a$$

Note that the second and third steps are bypassed in the G' derivation through the production $T' \rightarrow F'$.

Correspondence of G to G'

We now state and prove the theorem we asserted:

Theorem 5.1. A string “ $u x \# i$ ” is derivable in grammar G' if and only if “ uxv ” is a right-most sentential form in grammar G , and x is its handle.

Proof: “If” Part. We prove this part by induction on the number of derivation steps for uxv . For 0 steps, the sentential form in G is S , and S' is trivially derivable from S'' in G' .

For n steps, consider the last step:

$$uAv \Rightarrow uxv$$

By induction, the handle of uAv (wherever it is), augmented by some #i, is derivable in G' . Now the handle of uAv cannot lie entirely in u , since if it did, it would reduce to a nonterminal B to the left of A , violating the right-most derivation condition. Therefore the handle of uAv must include A or lie to right of A . (Note that the handle may be empty; the preceding sentence is true nevertheless). We then have two cases, as follows:

CASE 1.

$$u' B v' \Rightarrow u' r A s v' = u A v \Rightarrow u x v \quad \text{in } G.$$

Here the handle of uAv includes A . Let production $B \rightarrow rAs$ be number j . By induction, we have

$$S'' \Rightarrow^* u' B' \Rightarrow u' r A s \#j \text{ in } G'$$

But then we also have the production $B' \rightarrow rA'$ in G' , yielding

$$S'' \Rightarrow^* u' B' \Rightarrow u' r A' \Rightarrow u' r x \#i$$

where $A \rightarrow x$ is production number i .

CASE 2.

$$u A r B v' \Rightarrow u A r s v' = u A v \Rightarrow u x v$$

Here the handle of uxv is in v . Again, let production $B \rightarrow s$ be number j and production $A \rightarrow x$ be number i . By induction we have

$$S'' \Rightarrow^* u A r B' \Rightarrow u A r s \#j \quad \text{in } G'$$

In this derivation, there had to be a step that introduced A through a production

$$C' \rightarrow g A h$$

i.e.,

$$S'' \Rightarrow^* u' g A h h' \quad \text{in } G'$$

By the nature of the derivations in G' , h' must be empty and h contains a nonterminal in G' ; the string $u'gA$ is therefore uA . We also have an alternative production

$$C' \rightarrow g A'$$

in G' , hence

$$S'' \Rightarrow^* u A' \Rightarrow u x \#i.$$

QED

Proof of the only if part is left to an exercise.

LR(k) Augmented Grammar

Knuth [1965] develops a similar algorithm for the general case LR(k). Essentially, the nonterminals of G' are of the form $[A; x]$ where A is a nonterminal in G , $|x|=k$, and x is terminal and derivable from the string yv in a right-most sentential form uyv , where

$$S \Rightarrow^* uAv \Rightarrow uyv$$

The productions in G are then transformed into productions in G' through

the use of these special nonterminals and also through the special symbols $\#i$, in a manner similar to the above.

The grammar so obtained is a regular grammar, and can be shown to derive viable prefixes, together with the k -symbol lookahead. The FSA accepts a viable prefix and k symbols past it, then is in a state with a $\#i$ out-directed transition, where i indicates the production associated with the handle of the viable prefix.

Exercise

Write an augmented grammar for the context-free grammar

$$\begin{aligned} G &\rightarrow S \\ S &\rightarrow b D \\ S &\rightarrow E e \\ D &\rightarrow D d \\ D &\rightarrow \epsilon \\ E &\rightarrow E s \\ E &\rightarrow s \\ E &\rightarrow S \end{aligned}$$

then construct a reduced finite state machine from the augmented regular grammar.

6.4. Size of LR(k) Tables

Purdum [1974] gives a number of statistical formulas that yield the size of an LALR(1) parser reasonably accurately, without having to construct the parser. His study is based on 84 grammars, 65 of which had 15 or less terminals and nonterminals and the remaining 19 of which contained more than 15.

He found that the number of states S is a linear function of the grammar size G , as follows

$$S = bG + a$$

where

$$a = .02 \pm .45$$

and

$$b = .5949 \pm .0048$$

The number of transitions T is a linear function of a sum Q :

$$T = b'Q + a',$$

where

$$a' = 3.4 \pm 2.9$$

and

$$b' = .9867 \pm .0069$$

The small tolerances indicate that the number of states and transitions can be rather accurately estimated from the grammar. The grammar size G is the sum of the lengths (in tokens) of the right sides of the productions, plus the number of productions. Thus G is 18 for grammar G_0 .

The sum Q is more complicated to determine. We begin with the grammar, and augment each production with the special token $\#i$, where i is the production number. Then we represent the grammar as a set of trees. Each tree is defined on a nonterminal and contains all the augmented productions for that nonterminal. Any two productions of the forms

$$A \rightarrow xy$$

$$A \rightarrow xz$$

are represented by a common path from the root A that spells out the common prefix x . The suffixes y and z then are separate subtrees rooted in the right-most token of x . Thus, for the augmented grammar

$$S \rightarrow E \perp \#1$$

$$E \rightarrow E + T \#2$$

$$E \rightarrow T \#3$$

$$T \rightarrow F \uparrow T \#4$$

$$T \rightarrow F \#5$$

$$F \rightarrow a \#6$$

$$F \rightarrow (E) \#7$$

we have the trees shown in figure 6.8. Note that the tree edges are labeled with the production right-side symbols. The nodes are labeled with integer counts, determined as follows.

The count associated with a tree node is the size of the set $\{\text{FIRST}_1(A_1), \text{FIRST}_1(A_2), \dots, \text{FIRST}_1(A_n)\}$, where the transitions out of node N are labeled $A_1, A_2, \dots, A_n$. Thus in figure 6.8, the root node of the E tree has cardinality 5, since the union of the sets $\text{FIRST}_1(E)$ and $\text{FIRST}_1(T)$ is the set $\{E, T, F, a, ()\}$, of cardinality 5. The leaf nodes of the trees carry count 0, since they have no children.

Finally, the count Q is the sum of all the node counts for the production trees, less the counts for the root nodes, plus the count for the start root. Thus

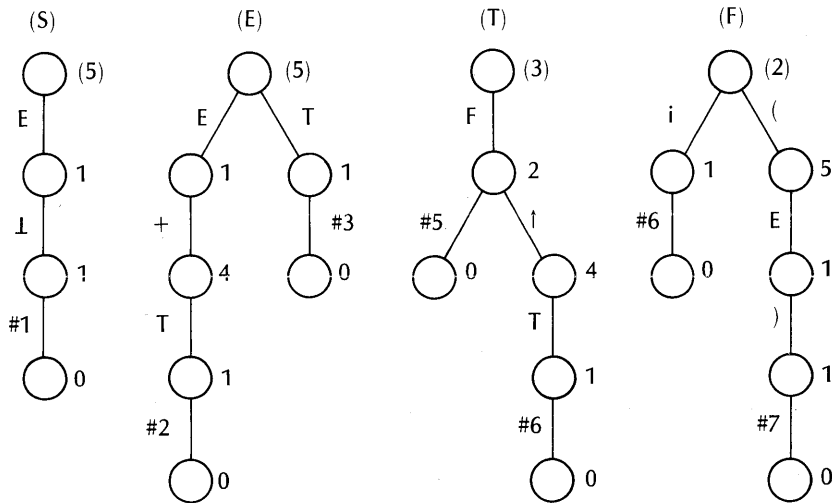


Figure 6.8. Production trees for Purdom's parser transition estimate.

we count only the start node root and all the remaining nonroot nodes. In figure 6.8, $Q = 29$.

Purdom's estimate for the number of transitions in this grammar is therefore $T = 31.0$. (The actual number of transitions is 30.) The estimated number of states $S = 12.51$, and the actual number is 13.

6.5. Comparison of Parsing Methods

The various parsing methods developed in this and the preceding chapters have important differences that have a direct bearing on the ease of construction of a viable production set, the construction of suitable semantics, the memory required for the parser, and the execution speed of the parser.

Generality

The LR(k) grammars and languages cover both the LL(k) and precedence grammars and languages. Thus, given an arbitrary grammar, the chances are greater that a LR(k) parser can be constructed for it than any other. Of LL(k) and the precedence grammars, little can be said. There are LL(1) grammars that are not simple (1,1) precedence and vice versa.

If we consider the class of extended grammars and their relation to recursive descent parsers, we find that a relatively straightforward extended grammar can be constructed for every language described by an LR(1) simple

grammar. Indeed, all the common programming languages have been implemented by recursive descent compilers, and such a compiler may be based on an extended grammar as we have seen. However, it is not clear that the LR(1) languages are equivalent to the 1-symbol extended grammar, recursive descent languages.

Recall that a non-LL(1) grammar can often be transformed into an LL(1) grammar. The practical consequence of such transformations is that an LL(1) parser has been constructed for several common languages, e.g., Algol (Lewis and Rosenkrantz [1971]). However, the transformed grammar invariably contains many more productions than an equivalent LR(1) grammar. The transformation is also not amenable to an algorithmic method, so that errors can be introduced into a given source grammar.

Of all the source grammar representations, the most easily understood is probably the extended grammar. Each right part is a regular expression and has a FSA representation that corresponds to a syntax graph of the language. One therefore has a very direct relationship between a very commonly used form of language definition and the basis grammar for an automatic parser generator.

Parser Size

Each of the parsers can be represented as a set of interpreted tables. Of all the parsing methods, the LALR parser tables are invariably smallest. For example, LaLonde [1971] gives the following comparative table sizes (in bytes) for four languages, and three different parsers: MSP, mixed-strategy precedence; WWSP, Wirth-Weber simple precedence; and LALR:

Grammar	Productions	MSP bytes	WWSP bytes	LALR bytes
XPL	109	3274	*	1250
EULER	120	3922	4321	1606
EULER-II	100	3017	3204	1276
ALGOL60	173	>6800†	>6100*	2821

* Not a WWSP grammar

† Not a MSP grammar

Comparative studies of LL(1) and LALR(1) parsers are not available, although there is reason to believe that they require comparable table space. The net space required by a recursive descent parser is, in our experience, about twice that required for an equivalent LALR parser.

Empirical Speed Comparisons

The comparison of parsing speed is subject to many variables only some of which depend directly on the parsing algorithm. Most modern computer systems employ a virtual memory, code segmentation, data segmentation, or some combination thereof. In such a system, only certain portions of the data or code is in fast memory at any instant; the bulk of the data and code is in auxiliary slow memory, e.g., a disk or drum. The execution speed of any program therefore depends critically upon such factors as (1) the amount of fast memory available, (2) the efficiency of the memory manager, (3) the access time of disk, and (4) the *working set* of the algorithm, that is, the amount of code and data that should be resident in fast memory for high performance.

An LALR or LL table-driven parser tends to require a rather large working set. It is difficult to choose a subset of the tables and code that would permit the parsing of a section of a program; in general, all the tables and the entire parsing algorithm are needed many times during the parsing of almost any source statement.

On the other hand, the working set of a recursive descent parser (not table driven) tends to be relatively small. The parsing of some statement may invoke relatively few procedures out of the entire set comprising the parser. For example, those procedures designed to parse the declarations would never be called while parsing the executable statements.

The semantic operations of a recursive descent compiler are often part of the parsing procedures. This practice tends to keep the working set small during compilation. The semantic operations of a table-directed parser must be separate from the tables, which means that the tables are of no value during code generation and vice versa.

Unfortunately, an empirical study of these matters has never been reported. Although the memory requirement for the LALR tables is smaller than for a recursive descent parser, it is by no means clear that an LALR-based compiler will be superior in overall performance.

LaLonde reports some performance comparisons for an MSP and an LALR parser for several grammars, as follows:

Program	Records	Tokens	Reductions	MSP secs	LALR secs
Compactify	77	439	1,262	0.84	0.52
LCS	3,322	17,369	58,707	28.86	15.88
XCOM	4,241	24,390	66,108	45.35	25.11
DOSYS	7,291	29,334	81,581	55.58	30.49

These results indicate a substantially greater efficiency for the LALR

parser. It is not immediately obvious just why this should be. The MSP parser uses fast indexing into a precedence matrix, based on two tokens, while the LALR parser must scan a list of tokens determined by its READ state. However, upon a reduction decision, the MSP parser must then scan a table of production right parts, comparing them with the stack contents, in order to determine the production, while the LALR parser yields the next production as part of the same function that locates the end of the handle. This latter saving apparently outweighs the cost of searching a READ state table.

6.6. Bibliographical Notes

The LR(k) parsing system is generally attributed to Knuth [1965], in which both the item-set construction and the augmented grammar construction are presented, with arguments for their correctness. He also proved that the LR(k) grammars are the largest class that can be deterministically parsed left-to-right with a k-symbol lookahead. Despite the significance of these results, very little additional work on LR parsers was done until more recently.

The SLR(k) resolution is from DeRemer [1969] and [1971]. The SLALR(k) resolution was first reported by LaLonde [1971a]. The LALR(k) system was first proposed by Anderson, Eve, and Horning (Anderson [1973]); it and the SLR(k) system seem to be the LR systems of choice for the known LR parser generators.

The optimizations of canonical LR(k) tables are from Aho [1972].

SYNTAX-DIRECTED TRANSLATION

We are conceptually at the midpoint of our subject. We have developed mechanisms for recognizing the structure of sentences written in the source programming language. The end result of this process of lexical analysis and parsing is a sequence of productions that constitute a bottom-up or top-down parse of the input. We may also view the end result (so far) as a derivation tree.

We have seen how a source string can be subdivided and analyzed according to an underlying context-free grammar and transformed into a structure (a derivation tree), in terms of which the underlying meaning of an input sentence can be defined.

We must now develop tools for the construction of the translation or object sentence. The output of any compiler can be viewed as a sentence in some object or target language, whether the object be another high-level programming language, symbolic assembly language, or sequences of binary machine instructions.

The general task of accepting the tree structure created by a parser and generating from it the object sentence (or code) is called the *synthesis* or *semantics* phase of a compiler. The synthesis phase can be conceptually subdivided into a number of reasonably distinct subtasks, as described in chapter 1, namely: collection and distribution of attributes of the data objects found in the abstract-syntax tree, optimization on the tree, allocation of data object space, and code generation.

7.1. General Principles

A *syntax-directed translation scheme*, or *SDTS*, specifies a source language, a target language, and rules for the translation of any string in the source language into a target string. The rules of an SDTS are production rules, extended to include translation forms. No mechanism for the translation of source to target is implied by the SDTS definitions, although we shall see that the translation can easily be achieved.

As a conceptual model, the SDTS provides a good framework for understanding some of the underlying principles of translation. An implementation of an SDTS is a *string translator*, but of a limited kind. The SDTS model does not contain provisions for collecting and distributing attributes, and without attributes a translator is of little practical use. Nevertheless, certain ordering properties stem from the SDTS model, and these are of considerable value in planning an attribute system for a complete compiler.

We shall first define the SDTS model and explore some of its properties. A general implementation will be presented. The chapter ends with the development of a general purpose translator system that can serve as the framework of a practical bottom-up compiler.

7.1.1. Definitions

A *syntax-directed translation scheme*, is a 5-tuple consisting of

1. A finite *input alphabet* Σ , comprising the symbols of the source language.
2. A finite set N of *nonterminal symbols* used in a context-free grammar to define the input language.
3. A finite *output alphabet* Δ , containing symbols that will appear in the *translation* or *output string*.
4. A finite set of *rules* R of the form $A \rightarrow w, y$, to be defined later.
5. A *start symbol* S in N with the same meaning and use as in the definition of a context-free grammar.

A rule in R of the form

$$A \rightarrow w, y$$

is such that “ w ” consists of a string of terminals and nonterminals, just as in a context-free grammar, and “ y ” is a string of symbols from N and Σ . The symbol A is in N . Furthermore, there must be a one-to-one association of nonterminals in “ w ” with the nonterminals in “ y ”. The string “ w ” is called the *source element* and the string “ y ” is called the *translation element* of this rule.

The 4-tuple (N, Σ, P, S) , where P is a set of rules of the form $A \rightarrow w$, and $A \rightarrow w, y$ is a rule in R , is called the *underlying source grammar* of T . That is, we obtain the underlying source grammar by stripping the translation element from the rules and discarding the output alphabet.

There is also an underlying *target grammar*, obtained by removing the source element from each production and discarding the input alphabet.

For example, consider the SDTS $S_1 = (\{E, T, A\}, \{a, b, c, +, -, [,]\}, \{\text{ADD, SUB, NEG, } x, y, z\}, R, E)$, where R consists of the set of translation rules

1. $E \rightarrow E + T, T \text{ E ADD}$
2. $E \rightarrow E - T, E \text{ T SUB}$
3. $E \rightarrow - T, T \text{ NEG}$
4. $E \rightarrow T, T$
5. $T \rightarrow [E], E$
6. $T \rightarrow A, A$
7. $A \rightarrow a, x$
8. $A \rightarrow b, y$
9. $A \rightarrow c, z$

The underlying source grammar of S_1 is

$$\begin{aligned} E &\rightarrow E+T \mid E-T \mid -T \mid T \\ T &\rightarrow [E] \mid A \\ A &\rightarrow a \mid b \mid c \end{aligned}$$

A *translation form* is a pair of strings (u, v) , such that “ u ” is a sentential form of the underlying grammar of an SDTS and “ v ” is a *translation* consisting of elements drawn from N and Σ .

A translation form is defined as follows:

1. (S, S) is a translation form, and the two S ’s are said to be *associated*. Also, S is the start symbol of the SDTS.
2. If $(a A b, a' A b')$ is a translation form, and the two A ’s are associated; further if $A \rightarrow g, g'$ is a rule in R , then $(a g b, a' g' b')$ is also a translation form. The association of nonterminals in g to those in g' must be carried into the translation form exactly as it is in the rule.

The notation

$$(a A b, a' A b') \Rightarrow (a g b, a' g' b')$$

expresses the transformation of one translation form into another.

We see that the first part of the translation form is exactly a sentential form of the underlying CFG grammar; the second part is an associated translation, a sentential form of the underlying target grammar.

The *translation* defined by an SDTS T , is the set of pairs

$$\{(x, y) \mid (S, S) \Rightarrow^* (x, y), x \in \Sigma^*, \text{ and } y \in \Delta^* \}$$

which is clearly analogous to the definition of a language in a context-free grammar given in chapter 2.

Example. Consider the input string

$$-[a+c]-b$$

that is derivable in the underlying grammar of S_1 as follows:

$$\begin{aligned} E &\Rightarrow ET \Rightarrow -T-T \Rightarrow -[E]-T \Rightarrow -[E+T]-T \\ &\Rightarrow -[T+T]-T \Rightarrow -[A+T]-T \Rightarrow -[a+T]-T \\ &\Rightarrow + -[a+c]-b \end{aligned}$$

The derivation tree for this string is shown in figure 7.1(a).

Now the translation for this string is given below. We have introduced subscripts on certain nonterminals in the translation forms as needed to indicate the association between the input and output nonterminals. Thus two T 's appear in the second translation form; they are distinguished by the subscripts 1 and 2.

$$\begin{aligned} (E, E) &\Rightarrow (E-T, E \text{ T SUB}) \\ &\Rightarrow (-T_1-T_2, T_1 \text{ NEG } T_2 \text{ SUB}) \\ &\Rightarrow (-[E]-T, E \text{ NEG } T \text{ SUB}) \\ &\Rightarrow (-[E+T_1]-T_2, T_1 \text{ E ADD NEG } T_2 \text{ SUB}) \\ &\Rightarrow + (-[A+T_1]-T_2, T_1 \text{ A ADD NEG } T_2 \text{ SUB}) \\ &\Rightarrow (-[a+T_1]-T_2, T_1 \text{ x ADD NEG } T_2 \text{ SUB}) \\ &\Rightarrow + (-[a+c]-T, z \text{ x ADD NEG } T \text{ SUB}) \\ &\Rightarrow + (-[a+c]-b, z \text{ x ADD NEG } y \text{ SUB}) \end{aligned}$$

The translation of “ $-[a+c]-b$ ” in the SDTS T is therefore:

$$z \text{ x ADD NEG } y \text{ SUB}$$

We evidently have a translator for some simple arithmetic expressions to a postfix notation, with the twist that pairs of addition operands are interchanged in the output string.

We have introduced some lexical operations in this grammar, through the last three translation rules: $A \rightarrow a, x \mid b, y \mid c, z$, in order to clarify the translation process. Of course, a set of identifiers will appear as one terminal symbol in a practical compiler and be distinguished by the lexical analyzer.

7.1.2. Tree Transformations

A tree interpretation of the syntax-directed translation process is shown in figure 7.1. Part (a) of this figure shows the derivation tree for the string “ $-[a+c]-b$ ” in the underlying grammar of T . The translation can be viewed as a transformation of this tree into another tree, the transformation consisting of (1) removing the terminal nodes, (2) permuting the children of each interior node according to the appropriate translation rule, and (3) adding terminal nodes that correspond to the translation terminal set Δ .

Thus figure 7.1.(b) shows the derivation tree in (a) with the terminal nodes removed. In this step, it is possible that two different productions can appear to be the same. For example, the productions

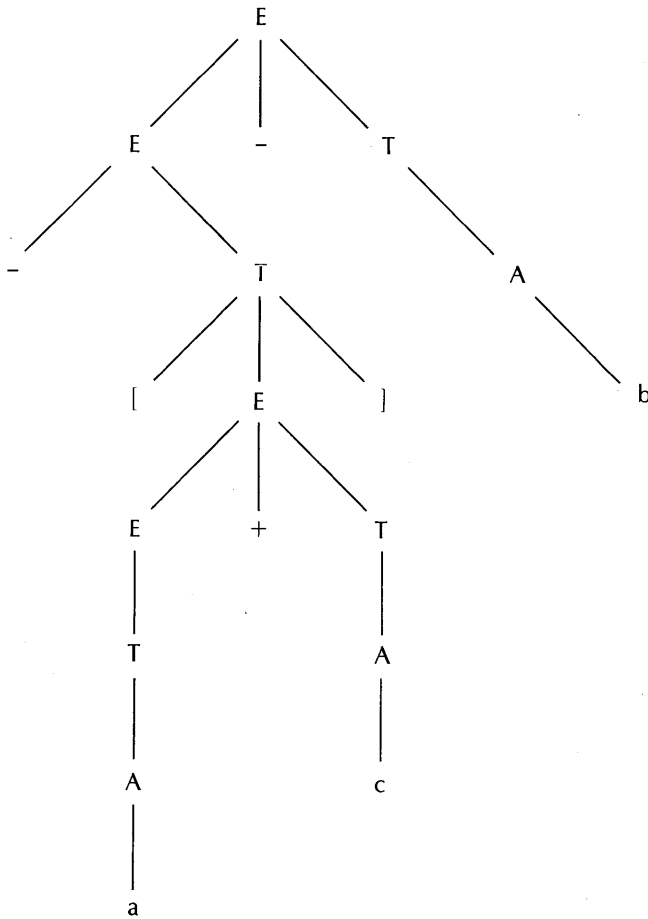


Figure 7.1(a). Derivation tree for $-[a+c]-b$.

$$\begin{aligned} E &\rightarrow -T \quad \text{and} \\ E &\rightarrow T \end{aligned}$$

look the same in the tree after the terminal nodes are removed. We therefore label each node with a production number.

In figure 7.1(c), the children of each node have been permuted according to the translation rules and the terminal output symbols have been added. Note that the children of each interior node now are exactly a translation element of some translation rule. For example, the children of node $_1E$ were originally

$$E + T$$

The translation rule is

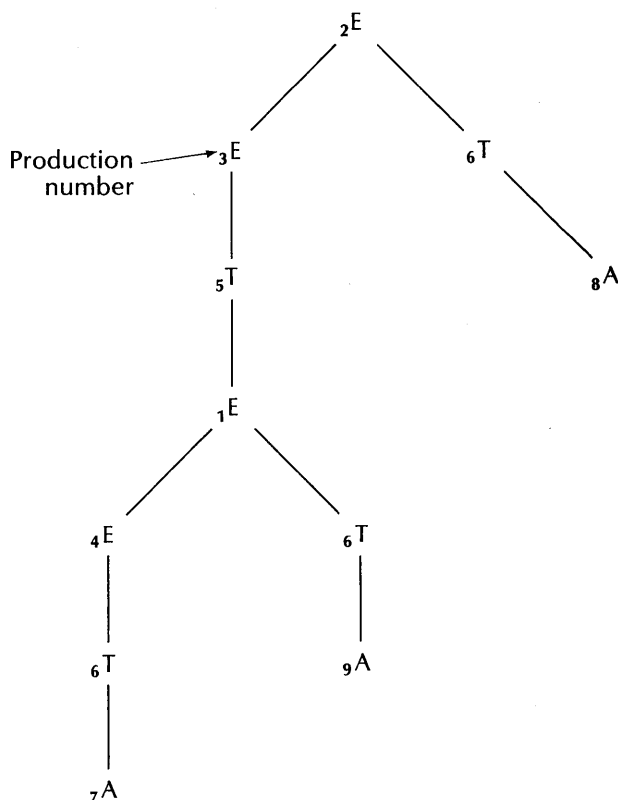


Figure 7.1(b). Derivation tree of figure 7.1(a) with terminal nodes removed.

$$E \rightarrow E + T, T E \text{ ADD}$$

hence the children of this node become

$$T E \text{ ADD}$$

T and E stand for a pair of subtrees that “move” with the permutation; this movement causes the string “z” to appear first in the translation although its corresponding source symbol “c” appears second in the source string.

The completed translation tree, figure 7.1(c) may now be scanned in left-to-right natural order to yield the translation string

$$z \times \text{ADD NEG } y \text{ SUB}$$

It is clear that the resulting translation string is independent of the order in which the translation is performed. Thus a bottom-up (right-most) or top-down (left-most) translation will yield the same translation string.

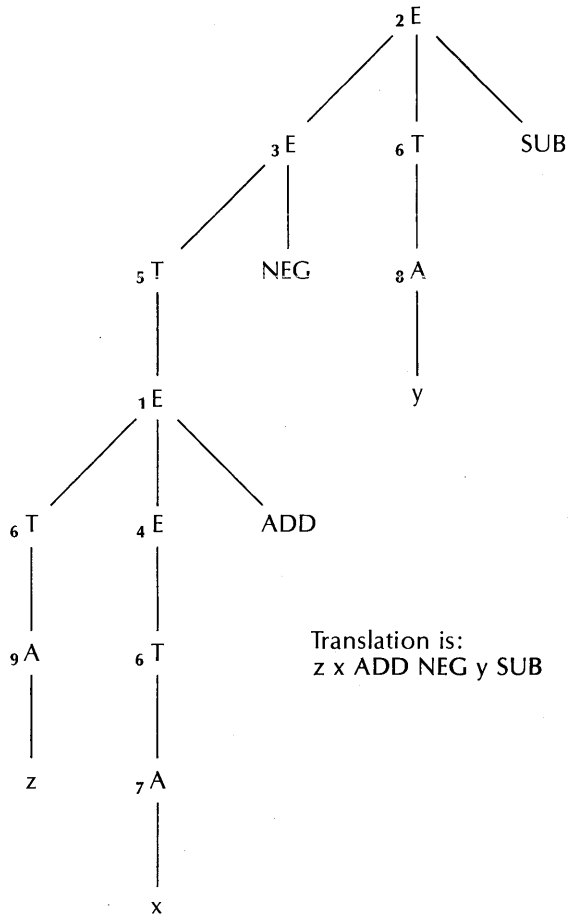


Figure 7.1(c). Derivation tree of figure 7.1(b) with nodes permuted and translation elements added, according to translation scheme.

Ambiguity

If the source grammar of a SDTS is ambiguous, then two or more derivation trees exist for some source string. Clearly, each of these derivation trees will yield a unique translation tree, however the two translations will in general be different.

We say that an SDTS is *ambiguous* if its source grammar is ambiguous and is *unambiguous* otherwise. It is clear that the addition of translation elements to a grammar cannot introduce or remove an ambiguity. However, it is possible that a translation can be unique despite a source grammar ambiguity; for example, if the translation element is identical to the source element, then every translation is unique despite an ambiguous source grammar.

Exercises

1. Develop a derivation and translation for the string

$$-c + [-b - a]$$

in SDTS S_1 .

2. Prove that if the underlying source grammar G of an SDTS is unambiguous, then the translation string of any source string in $L(G)$ is unique.
3. Given an SDTS S , how may an inverse SDTS S' be defined? (If S translates a string "w" into "w'", then S' translates "w'" into "w".)

7.2. Simple SDTS and Top-Down Transducers

A SDTS is said to be *simple* if the order of the nonterminals in the translation part of each rule is the same as the order of the nonterminals in the source part. For example, a rule

$$S \rightarrow S_1 + S_2, S_2 S_1 \text{ ADD}$$

in a SDTS T causes T to be nonsimple.

A simple SDTS is such that no permutation of the derivation tree nodes is required for the translation. We have only removal of the source string terminals and insertion of the translation string terminals.

The significance of a simple SDTS for a top-down parser is expressed in the following theorem:

Theorem 7.1. If $T = (N, \Sigma, \Delta, R, S)$ is a simple SDTS whose underlying grammar is $LL(k)$, then there exists a top-down deterministic push-down transducer that accepts any string in the input language of T and whose yield is the corresponding output string.

A *push-down transducer* (PDT) is analogous to the push-down automaton of chapter 4, except that it is permitted to emit a string of finite length (the translated output) upon each move. We define a push-down transducer P as follows:

A *configuration* of P is a 4-tuple (q, x, y, z) , where q is a state in its finite control, x is the remaining input list yet to be scanned, y is a stack, and z is an output string emitted so far up to this point. Then in one move, we have

$$(q, ax, Yy', z) \vdash (r, x, gy', zz')$$

where there exists a machine rule

$$\delta(q, a, Y) \text{ contains } (r, g, z')$$

That is, given state q , with input symbol a , and stack top symbol Y , the machine is permitted to move to state r ; in this move, the input symbol a is discarded, Y on the stack top is replaced by g , and the string z' is emitted as output.

Then we say that w is an *output* for x if $(q_0, x, Z_0, \epsilon) \vdash^* (q, \epsilon, u, w)$ for some state q and stack string u . State q_0 is the initial state and Z_0 is the initial stack contents; ϵ is the empty string. We say that P *halts by empty stack* if $u = \epsilon$ is a halting condition; alternatively, P *halts by final state* if q is a member of a defined set of halt states F , a subset of the machine's state set. These definitions are analogous to those of the PDA's defined in chapters 4 and 5.

We say that the PDT P is deterministic when both of the following conditions are met:

1. For all states q , strings a , and stacks Z , $\delta(q, a, Z)$ contains at most one element.
2. If $\delta(q, \epsilon, Z)$ is not null, then no symbol " a " exists such that $\delta(q, a, Z)$ is not null. That is, there must be no conflict between an empty and a nonempty move for some state and stack.

We may now describe the construction of a PDT P , given a SDTS T , where the underlying grammar of T is $LL(1)$, such that P : (1) accepts every string in the underlying grammar of T , and (2) emits exactly the translation in T of every such string. The general case of $LL(k)$, $k > 1$, is discussed in Lewis [1968].

Recall, from chapter 4, that the top-down $LL(1)$ recognizer has two kinds of move:

1. An *apply* move, in which a nonterminal A on the stack top is replaced by a string " w ", where $A \rightarrow w$ is some production rule. This operation is made deterministic through a selection table based on the stack top nonterminal A and the next input symbol.
2. A *matching* move, in which a terminal symbol " a " on the stack top is matched against the next input symbol. In a matching move, the stack top symbol is removed and the read head is advanced one symbol. A failure to match must be a syntax error.

Now a PDT operates as follows:

1. In an apply move, with a nonterminal A on the stack top, we know that a production $A \rightarrow w$ is involved, from the deterministic selector table. Let the translation rule in R be

$$A \rightarrow w, z$$

where

$$w = a_0 B_1 a_1 B_2 a_2 \dots B_k a_k$$

and

$$z = b_0 B_1 b_1 B_2 b_2 \dots B_k b_k$$

The B_i are nonterminals, the a_i are input alphabet strings (or empty), the b_i are output alphabet strings (or empty), and $k \geq 0$. Note that this translation rule is "simple."

We assume that the input and output alphabet symbols are distinguishable. Then A is replaced on the stack top by the composite string

$$b_0 a_0 B_1 b_1 a_1 B_2 \dots B_k b_k a_k$$

with b_0 on the stack top.

2. If the symbol on the stack top is a member of the output alphabet, then it is removed and emitted as output.

3. If the symbol on the stack top is a member of the input alphabet, then it is matched against the input string (syntax error otherwise), it is removed from the stack, and the read head is advanced one position.

Example. Consider the simple SDTS

$$\begin{aligned} S &\rightarrow 1 S 2 S, x S y S z \\ S &\rightarrow 0, w \end{aligned}$$

Note that we do not need to superscript the S 's, since the SDTS is simple. The underlying grammar is obviously LL(1). We shall define the push-down transducer and trace its operation for an example input string.

The transducer:

1. On stack top S , if the input symbol is
 - (a) "1" then pop " S " and push " $x1Sy2Sz$ "
 - (b) "0" then pop " S " and push " $w0$ "
2. On stack top in $\{x, y, z, w\}$, emit the stack top as output.
3. On stack top in $\{0, 1, 2\}$, match against the next input symbol.

A machine trace for sentence 1102020:

Operation	Remaining input	Output	Stack
Initially	1102020		S
1a	1102020		$x1Sy2Sz$
2	1102020	x	$1Sy2Sz$
3	102020		$Sy2Sz$

1a	102020		x1Sy2Szy2Sz
2	102020	x	1Sy2Szy2Sz
3	02020		Sy2Szy2Sz
1b	02020		w0y2Szy2Sz
2	02020	w	0y2Szy2Sz
3	2020		y2Szy2Sz
2	2020	y	2Szy2Sz
3	020		Szy2Sz
1b	020		w0zy2Sz
2	020	w	0zy2Sz
3	20		zy2Sz
2	20	zy	2Sz
3	0		Sz
1b	0		w0z
2	0	w	0z
3	ε		z
2	ε	z	zε

The output string is therefore “xxwywzywz”. Note that the transducer accepts by empty stack.

If we did not have an LL(1) selector table, we would have a nondeterministic PDA. We have seen in chapter 4 that a PDA that accepts any context-free grammar can be constructed. We need only extend the notion of nondeterminism to a transducer, and this is easily done. Let *acceptance* of an input string mean that some set of choices based on stack top symbols eventually leads to an empty stack and input list. Then the emitted output is a translation.

If the underlying grammar is ambiguous, then no deterministic PDT exists, although a nondeterministic PDT exists; there may then exist more than one translation for one or more input strings.

If the underlying grammar is LL(1) then it is unambiguous, and there is exactly one translation for every acceptable input string.

The determinism of an LL(1) transducer means that the output can be immediately printed or punched on each move; it need not be saved on some erasable tape until the entire parse is complete. Output can also be generated as soon as a production is identified; the translator need not sustain output until the entire right part of the production is completely associated with the input list. Of course, it obtains this predictive ability through the one-symbol lookahead—the next symbol in the input list must be an infallible guide to the production rule about to come up, or else the grammar is not LL(1).

Among other things, determinism means that an entry in a symbol table could be made at the beginning, in the middle, or at the end of a production. Although we technically are discussing string-to-string translators, certain output string elements can in fact carry other kinds of operations, such as:

“enter the preceding identifier in a symbol table,” or “turn off the compiler listing flag,” etc. Such actions may be difficult or impossible to undo and must also be performed at the right time in the compilation. Such timing questions can be resolved for a LL(1) push-down transducer through a study of the production tree structure and the position of the action in some production.

Exercises

1. Trace the above transducer for the string 1021020.
2. Show that the PDT emits a correct translation for any acceptable input string.
3. Write a Pascal LL(1) translator. The program should include suitable data structure definitions.

7.3. Simple Postfix SDTS and Bottom-Up Transducers

A SDTS is said to be *simple postfix* if it is simple, and, in addition, every translation rule has the form

$$A \rightarrow a_0 B_1 a_1 B_2 a_2 \dots B_k a_k, B_1 B_2 \dots B_k w$$

That is, no translation terminals may appear in the translation element except as the right-most string “w”. The significance of a simple postfix SDTS is expressed in the following theorem:

Theorem 7.2. For every simple postfix SDTS whose underlying grammar is LR(k), there exists a deterministic LR(k) push-down transducer that (1) accepts every sentence derivable in the underlying grammar and (2) emits as output the translation of that sentence.

Recall from chapter 6 that the LR(k) parser has three actions, selected by a current state and a next symbol, as follows:

1. *READ action:* the next input symbol must be among the terminal symbols associated with this state; if so, then advance the read head, push the next state S' and go to state S' .
2. *APPLY action:* the stack top contains a set of states associated with the handle of some production i . The production number is specified by this state. Pop the states corresponding to the handle, push a state S' determined from the exposed stack top state and the GOTO table, then go to state S' .

3. *ACCEPT action*: a goal-symbol production has just been reduced, and the stack contains one state corresponding to the goal symbol. Halt, with an indication that the input string is accepted. (We assume that the goal symbol never appears in the right-hand part of any production.)

These operations are directed by a finite-state control, with one or more states, that is permitted to examine as many as k symbols in the input in determining its action.

This machine may be easily transformed into a postfix transducer, by adding the following operation to the APPLY rule:

Emit the terminal portion of the translation element associated with rule i .

We leave machine traces and a proof of theorem 7.2 to the exercises.

Theorem 7.2 has an important converse, as follows:

Theorem 7.3. There are unambiguous simple SDT's with an underlying LR(k) grammar that cannot be translated by any deterministic push-down automaton. (Note that this theorem has no LL(k) counterpart.)

This theorem emphasizes the importance of the postfix condition. For example, consider the SDTS T with the rules

$$\begin{aligned} S &\rightarrow Sa, xS \\ S &\rightarrow Sb, yS \\ S &\rightarrow \epsilon, \epsilon \end{aligned}$$

These rules are simple, but not postfix. It is easy to show that the underlying grammar is LR(1); however, no deterministic push-down transducer can be devised that will correctly translate its input language. Intuitively, it is necessary to emit an "x" or a "y" before the transducer can determine which of the three productions applies. Let us go through an example.

Consider the string "ba", which translates to "xy" (see figure 7.2.) A trace of the LR(1) machine apply states looks like this:

Stack	Input List	Production
ϵ	ba	$S \rightarrow \epsilon$
Sb	a	$S \rightarrow Sb$
Sa	ϵ	$S \rightarrow Sa$

To emit the required string "xy", the machine must somehow predict that production $S \rightarrow Sa$ will eventually appear. What it in fact reports as apply states are the other two productions first.

1. $S \rightarrow Sa, xS$
2. $S \rightarrow Sb, yS$
3. $S \rightarrow \epsilon, \epsilon$

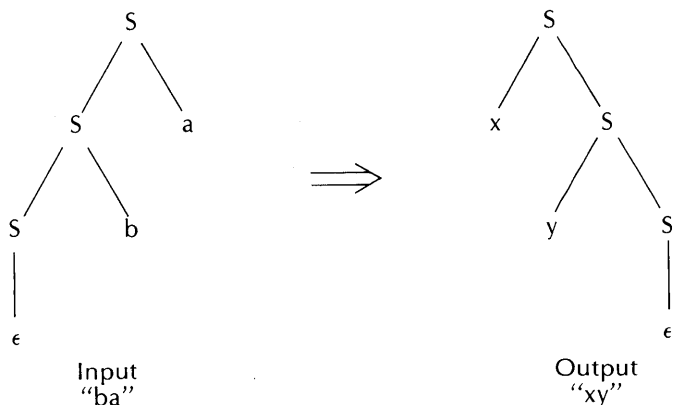


Figure 7.2. Translation of a string "ba" in a non-postfix simple syntax-directed translation scheme.

Now a top-down automaton could conceivably emit an "x" first, based on its start state and the first k input symbols. The top-down derivation of "ba" begins with $S \rightarrow Sa$; and "x" could therefore be emitted. However, this grammar is not $LL(k)$ for any k ; it contains left-recursive productions. This means that, for whatever k we choose, there is a string in the source language that will require a lookahead of at least $k + 1$ in order to determine the next production in a top-down manner. We conclude that no deterministic transducer exists for this SDTS; the information that it needs in order to emit a translation can be arbitrarily far ahead in the input string.

It would appear that a simple postfix SDTS is more restrictive than a simple nonpostfix SDTS, and that a top-down SDTS is therefore somehow more powerful than a bottom-up SDTS. Such is not the case; this view overlooks the possibility of modifying the grammar in order to achieve the effect, if not the substance, of a simple nonpostfix SDTS.

Suppose that we want a translation output y within a translation element:

$$A \rightarrow xz, XyZ$$

where the rule $A \rightarrow xz, XZ$ is simple postfix and Z is nonempty. We may then consider the pair of productions

$$\begin{aligned} A &\rightarrow x Y' z, X Y' Z \\ Y' &\rightarrow \epsilon, y \end{aligned}$$

Note that both productions are simple postfix. If the underlying source

grammar is still LR(k) with this change, then a bottom-up deterministic transducer will generate the desired translation. If the grammar is no longer LR(k), then it can be shown that the original grammar is not LL(k).

Another way to transform the productions is as follows:

$$\begin{aligned} A &\rightarrow X' z, X' Z \\ X' &\rightarrow x, X y \end{aligned}$$

Here, we have assigned the string “x” to a new production, permitting the postfix translation Xy ; the A production then becomes postfix as well.

The restriction to “simple” in both the bottom-up and top-down transducers is also easily removed by a system based on the SDTS transducers, as we shall see in section 7.4. Essentially, we may simply generate derivation trees (or better, abstract-syntax trees) and then rearrange the nodes internally as needed to achieve the necessary non-simple translation. Such a scheme may be limited in practice by the amount of random access storage space available for the trees; however, most modern computer systems are free of such limitations.

Examples. Consider grammar G_0 augmented with postfix translation elements given below:

$$\begin{aligned} E &\rightarrow E + T, E T \text{ ADD} \\ E &\rightarrow T, T- \\ T &\rightarrow T * F, T F \text{ MPY} \\ T &\rightarrow F, F \\ F &\rightarrow A, A \text{ LOAD} \\ F &\rightarrow (E), E \\ A &\rightarrow A a, a \\ A &\rightarrow a, a \end{aligned}$$

This SDTS is clearly simple postfix. A translation of the expression

$$aa * (aaa + a)$$

is easily shown to be

$$aa \text{ LOAD } aaa \text{ LOAD } a \text{ LOAD ADD MPY}$$

and is exactly that required by a postfix stack machine. The LOAD operation operates upon the preceding identifier. If we wish LOAD to operate on the following identifier instead, we may modify one of the F productions as follows:

Instead of production $F \rightarrow A, A \text{ LOAD}$, we introduce the two productions

$$\begin{aligned} F &\rightarrow L A, L A \\ L &\rightarrow \epsilon, \text{LOAD} \end{aligned}$$

The LOAD will clearly then precede its identifier. The grammar is still LR(1); the reduce operation for the empty production $L \rightarrow \epsilon$ can be inferred from a one-symbol lookahead of the following identifier.

The translation of $aa*(aaa + a)$ will then be

LOAD aa LOAD aaa LOAD a ADD MPY

As another example, consider the common control statement

$S \rightarrow \text{if } E \text{ then } S \text{ else } S$

where E is an expression and S is a statement. In this form, a simple postfix translator can only generate evaluation code for E , then evaluate the two statements S . Only when both of these are complete is it known that an if-then-else control statement is being parsed. But such a statement requires the emission of a conditional branch instruction between the E and the first S , and an unconditional branch instruction between the two S 's. How might this be done?

One solution is to partition this production into three productions:

$S \rightarrow I \text{ else } S$
 $I \rightarrow T \text{ then } S$
 $T \rightarrow \text{if } E$

Clearly, the three put together are equivalent to the original production. However, by so separating the three productions, we may generate a simple postfix translation as follows:

$S \rightarrow I \text{ else } S,$ $I S; L2:$
 $I \rightarrow T \text{ then } S,$ $T S; BR L2; L1:$
 $T \rightarrow \text{if } E,$ $E; BRF L1$

Here, "BR L2" means "branch to the statement labeled L2." "BRF L1" means "branch to L1 if the stack-top expression is FALSE, otherwise continue. Delete the stacktop in any case."

The semicolons will appear as separators in the translation strings.

Then the statement

if a then S_1 else S_2

would translate to

LOAD a ;
 BRF L1;
 S_1 ;
 BR L2;

L1: S2;
L2:

Here, L2 marks whatever statement follows the if-then-else.

Given the same if-then-else structure, we may achieve the same output by introducing productions for the “then” and “else” keywords, e.g.

$S \rightarrow \text{if } E \text{ H S L S ,}$	$E \text{ H S L S ; L2 :}$
$H \rightarrow \text{then ,}$	; BR L2 ; L1 :
$L \rightarrow \text{else ,}$	; L2 :

The effect of the added productions is clearly to bring out some translation strings at the desired point in the parsing of the whole statement.

Finally, empty productions may be used:

$S \rightarrow \text{if } E \text{ then H S else L S ,}$	$E \text{ H S L S ; L2 :}$
$H \rightarrow \epsilon ,$	; BR L2 ; L1 :
$L \rightarrow \epsilon ,$	; L2 :

These three translation schemes are equivalent and are LR(1), assuming that the remainder of the grammar is reasonable.

As an example of one limitation of a simple postfix SDTS, consider a function call, with the underlying source grammar:

$F \rightarrow A (L)$
 $L \rightarrow L ; E$
 $L \rightarrow E$
 $A \rightarrow A a$
 $A \rightarrow a$

The call parameter list L consists of parameters E separated by semicolons. The usual separator is a comma, but we are using a comma as a metasymbol.

For discussion purposes, we would like the procedure call to appear at the end of the code for the procedure parameters; this is how procedures are called on many (but not all) machines. However, the procedure name appears first in the syntax. We therefore need a nonsimple transducer if the call and the call name is to appear after the actual parameter coding.

However, a postfix call may be acceptable on certain machines, and is easily generated by the simple postfix grammar:

$$\begin{aligned}
 F &\rightarrow A (L) , A L ; \text{CALL} \\
 L &\rightarrow L ; E , L E \\
 L &\rightarrow E , E \\
 A &\rightarrow A a , A a \\
 A &\rightarrow a , a
 \end{aligned}$$

Then the function call

$$aa(a, aaa + aaaa)$$

will translate to

$$aa; \text{LOAD } a; \text{LOAD } aaa; \text{LOAD } aaaa; \text{ADD}; \text{CALL}$$

and the CALL operates on the very first identifier, aa. Nothing can be done using a simple SDTS to emit the procedure name “aa” after its CALL.

7.4. A General Transducer

A simple SDTS can be implemented by a push-down stack automaton, augmented with a translation string emitter, as we have shown in the preceding two sections. A non-simple SDTS is easily implemented by constructing a rearranged tree for the translation string as part of the parsing stack operations. When the parse is complete, the translation tree may be scanned in natural order to yield the translation string. Of course, the resulting machine is no longer a PDA.

The system presented in the following algorithm assumes that the base grammar is LR(k). However, the extension to any base grammar or to other parsers is trivial.

Let M be a translation machine that carries two stacks, one the usual LR(k) viable prefix stack (the *parse* stack) and the other a stack of translation subtrees (the *translation* stack). During the parsing process, each translation subtree will correspond to a nonterminal in the parse stack, and will represent a translation tree valid for that nonterminal and the derivations that previously stemmed from it. The translation stack will carry the roots of translation trees. Each nonterminal in the parse stack corresponds to a root in the translation stack.

Machine M has the actions READ, APPLY, and ACCEPT as follows. (The ERROR action in an LR parser is of no special interest just now.)

1. *READ*: The next input symbol is shifted into the parse stack. The translation stack is unaffected. A terminal in the parse stack does not have a corresponding translation stack element.
2. *APPLY*: Suppose we have the translation rule

$$N \rightarrow a_0 A_1 a_1 A_2 \dots A_m a_m, w_0 B_1 w_1 B_2 \dots B_m w_m$$

Remove the string " $a_0 A_1 \dots A_m a_m$ " from the parse stack, but not the associated subtrees (yet). The translation stack is associated with trees rooted in the A_i . Remove these trees from the translation stack and permute them according to the translation rule; they now correspond to the B_i . Create a new tree root N' and new tree leaves for the translation elements w_i and arrange these two sets of trees as the children of the new node N' . Now push N' into the translation stack.

At the end of the APPLY step we have a new subtree rooted in N' . The children of N' consist of leaves and subtrees, corresponding to the elements of the translation part of the rule, i.e., to $w_0 B_1 w_1 \dots B_m w_m$. The B_i have been obtained from the translation stack and the w_i have been newly created.

3. *ACCEPT*: The tree associated with start symbol S in the parse stack is the translation tree. It may be scanned in left-to-right natural order to yield the translation string.

This translator is illustrated in figure 7.3 for the nonsimple SDTS

1. $S \rightarrow aSA, 0AS$
2. $S \rightarrow b, 1$
3. $A \rightarrow bAS, 1SA$
4. $A \rightarrow a, 0$

and the input string "abbab". The underlying grammar is SLR(1). Only the apply steps are shown. For example, just before the first apply action, the stack contains "ab" and the input is "bab". Rule 2, when applied, causes "b" on the stack top to be replaced by "S", linked to a son "1". String "ba" then is shifted into the stack in preparation for the next apply action.

Just before the last apply step, the parse stack contains aSA and the translation rule is

$$S \rightarrow aSA, 0AS$$

A new leaf is created for the 0 of the translation element, and the A and S trees are interchanged. The stack top trees are replaced by a tree representing the translation subtree corresponding S , in this case, the complete translation tree. This tree may clearly be scanned from left to right to yield the translation string "01101".

A practical tree structure suited to this algorithm is illustrated in figures 7.4 and 7.5. Each translation stack element is a pointer to a *tree stack* that in turn carries a *mark*, a *token* and a *child* within each element. The children of some node are arranged contiguously in the tree stack, and only the right-most child carries a mark, indicating that it is the last child. If a tree node N is the root of a subtree, then N carries a child pointer, otherwise its child pointer cell is empty.

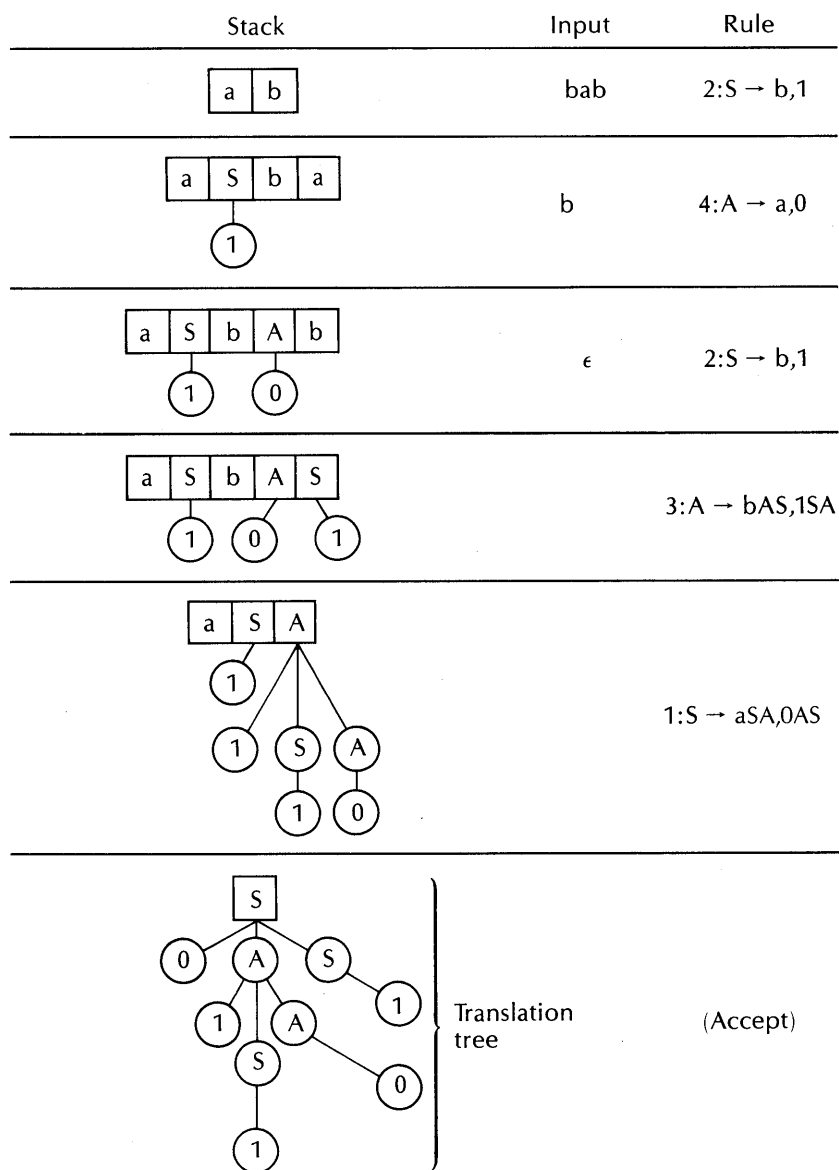


Figure 7.3. Translation by tree construction. Only the apply steps are shown.

Thus, in figure 7.4, the tree rooted in S has the children “0AS”, in tree stack nodes 6, 7, and 8. Node 7 is the root of a subtree in which level 1 is “1SA”. Figure 7.5 shows the tree structure represented by figure 7.4.

An empty leaf node must be specially denoted by a null token, since a leaf

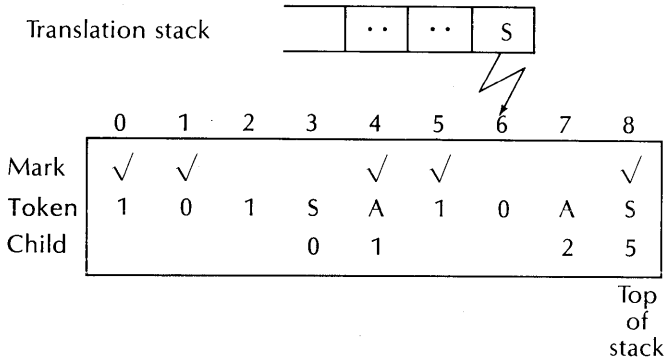


Figure 7.4. A compact representation of the tree of figure 7.4.

must occupy at least one cell to carry a mark.

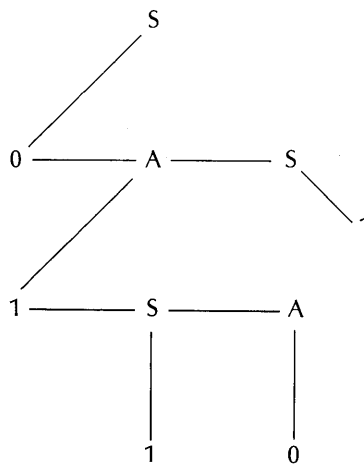
This representation is easily generated during translation, since all the children of some node can be allocated at once in the same APPLY operation and may therefore be contiguously arranged without any movement of nodes in the stack.

We may therefore modify the tree building algorithm as follows:

Introduce a third stack of three elements each: a mark, a token, and a child pointer; call it the *tree* stack. We really don't need the token, but we shall eventually want to use it to associate more information with the translation tree. Then modify the APPLY step as follows:

2. *APPLY*: Given the translation rule

$$N \rightarrow a_0 A_1 a_1 A_2 \dots A_m a_m, w_0 B_1 w_1 B_2 \dots B_m w_m$$



the translation stack will contain pointers to subtrees corresponding to $A_1, A_2, \dots, A_m$. Augment the tree stack with sufficient elements to contain all the symbols in the translation part $w_0 B_1 \dots B_m w_m$. Mark the top of the tree stack. Set the pointers associated with the w_i to null. Set the pointer associated with a node B_i to the pointer in the translation stack corresponding to A_j , where A_j is associated with B_i in the translation rule. Finally, remove the top m elements from the translation stack, and replace it with a pointer to the left-most new element in the tree stack.

Exercises

1. Devise a simple postfix SDTS for each of the following structures. V is a variable, E is an expression, S is a statement, and L is a statement label.

$$\begin{aligned} S &::= \text{while } E \text{ do } S \\ S &::= \text{for } V:=E \text{ until } E \text{ do } S \\ S &::= L: S \\ S &::= \text{case } E \text{ SL endcase} \\ SL &::= SL ; S \mid S \quad \{\text{statement list}\} \end{aligned}$$
2. Write a Pascal program that implements the apply step of the general transducer given in the previous section.
3. Devise data structures to facilitate a table-driven PDT that generates a translation tree.
4. State an SDTS implementation for an LL(1) source grammar, based on a tree builder.

7.5. String Transducers and Their Limitations

We have seen how an SDTS transducer can be applied as a string-to-string translator. Such a translator has certain practical applications, especially if extended by incorporating symbol table functions and some simple arithmetic. Some compilers are essentially string translators, from a high-level source language to symbolic assembly language for a machine. The advantage of writing such a compiler is its very low cost. The translator can be written as an SDTS and interpreted by a simple general purpose system.

We have also seen that an SDTS transducer can be used to construct a translation tree directly (section 7.4). We may modify that algorithm and generate an abbreviated tree, called an *abstract-syntax tree*; this notion will be expanded upon in section 7.5.2.

Finally, rather than simply emit a string, certain string tokens or all the tokens can be interpreted as synthesis actions.

7.5.1. String Translators

Let us assume that a general SDTS transducer is available, such as that described in section 7.4. What sorts of translations can be achieved and what are its limitations?

A great deal of literature exists on translator writing systems, or compiler-compilers, e.g., Feldman [1968] or, more recently, Griffiths [1974c]. A translator writing system, or TWS for short, often contains a string translator as its core, but is augmented by a variety of other needed functions.

Roughly speaking, a string translator can accept any source language defined by a context-free grammar, preferably LR(1), and emit a translation defined by the SDTS structure. The translation language may bear scant resemblance to the source language. Metcalfe [1964], for example, illustrates a translator of some simple English sentences to German, an operation that requires some rearrangement of sentence structure.

Unfortunately, the problem of language translation is not so easily solved. Natural languages are not context-free, nor are the common computer artificial languages. Both require some auxiliary mechanisms, in the form of a symbol table or a dictionary, at least, to achieve a reasonable translation.

Let us examine some of the translation needs that cannot be satisfied by a pure string translator.

Data Types

Data variables are usually typed, e.g.,

var I: integer; var R: real;

The effect of such statements is to associate one or more attributes with the identifiers I and R, as well as allocate space for them. Now a variable reference, such as:

R := I + 1;

usually requires knowledge of the variable types to generate correct code. Thus knowing that I is type "integer" and R is type "real," a compiler might generate the assembly code

LOAD	I	{fetch variable I to top of stack}
LOAD	=1	{push a 1 into stack}
ADD		{e.g., integer add}
FLT		{convert to real}
STD	R	{store double value on stack top in R; the "float" converts from a single to a double value}

for a stack machine. The choice of each instruction in this sequence depends on the types of the two variables.

Typing is achieved through a token table mechanism, and without some extensions to a string translator, we cannot hope to generate suitable translation sequences that consider variable typing.

However, typing can be bypassed in one of these ways:

1. The language may have only one type, for example, "real," so that every operation is determined only by the algebra and not the variables.
2. The machine representation of a data item may contain a data type descriptor along with the data item. Then all the necessary conversions and choice of the appropriate operation becomes a machine function and not a compiler function. For example, a single generic "add" instruction might be interpreted by the machine as a "real" add, an "integer" add, etc., depending on the operand descriptor.
3. The translation may be to a high level language that supports the features of the simpler language. The syntax may be different, but if the object language contains all the features and semantics of the source language, then a straightforward string translation is feasible.

However, variable typing occurs in most programming languages, and must usually be dealt with.

Array Dimensions

Consider an array declared as follows:

array X[0:25, 0:35] : real;

A reference to this array of the form X[I, J] may require knowledge of one or the other dimensions, depending on the storage conventions for multidimensional arrays in the target machine system. For example, the array may be stored in rows in one long linear array. An offset N of the form

$$N = 36 * J + I;$$

must then be computed for the reference X[I, J]; the "36" is the second dimension plus 1 (because of zero basing). Again, a simple string translator cannot supply the "36" without some help.

Nevertheless, there are ways:

1. Given an array declaration, a string translator can create a constant declaration, by making up new names based on the old ones, e.g.

source: array X[0:25, 0:36] of real;

object: array X[(25-0+1)*(36-0+1)] of real;

const #X0 = 35-0+1;

Then the new name #X0 can be used in a variable reference of the form X(J, I) in place of the needed dimensional factor 36:

$$N := \#X0 * J + I;$$

However, this approach to array indexing supposes that the object language translator can support a class of manufactured names and constant expression structures, which is seldom the case.

2. An array might be supported by a *dope vector* at run-time that is associated with the array elements and that contains the array dimensions. Then an array reference can be handled by a call to some general purpose procedure that accepts the indices I and J and a reference to the dope vector.
3. A multidimensioned array could be accessed through a transfer vector system. Consider a reference to X[I, J] again, and let there be a vector of 36 pointers (corresponding to the 36 possible J values). Each of the 36 pointers points to an I vector, e.g., one slice of the array. Then a fetch of X[I, J] could be coded without knowing the J dimension: indirect through V(J), then indexed by I. For some machine architectures, this kind of array access may be considerably more efficient than one based on a calculation of a linear array index.

Despite these solutions to array indexing, there is really no practical alternative to a symbol table system.

Branches and Procedure Calls

A string translator can only organize branches and procedure calls symbolically; without arithmetic or a token table, it cannot fix any program addresses. Furthermore, a string translator cannot ascertain whether a given branch label or procedure name has appeared more than once, so that an error of this kind cannot be reported until the object translation occurs.

Parameter Passing

The parameters of a procedure are usually of several different kinds (reference, value, name, etc.). Thus the generated code for the actual parameters depends on the declarations of the formal parameters. This dependency is another area of deficiency of a simple string translator—it must somehow generate object strings for actual parameters that are acceptable regardless of the declarations of the corresponding formal parameters, which may be impossible to do.

Some languages, such as Fortran, have only one passing mechanism—reference. In that case, a string translator can generate acceptable code.

In any case, an error in the number of actual parameters cannot be detected until the object is translated.

If a function call and an array reference appear syntactically the same, the object language of a string translator must be indifferent to the two. Thus in the Fortran statement

$$Y = X(I, J)$$

the $X(I, J)$ could be an array reference or a function call. The two cases are distinguished through a prior declaration of X as an array or a function. A symbol table is necessary to associate such an attribute with each identifier.

7.5.2. Abstract-Syntax Tree Construction

An *abstract-syntax tree* or AST is a condensed tree representation of some language structure. It contains only that information needed for the remaining transformations or reductions of the structure. Any language structure can potentially be represented as an AST, e.g., expressions, control statements, input or output statements, and declarations.

An AST can serve as an intermediate structure in a partitionable compiler. The compiler system that generates it might be one distinct piece of software, and the system that interprets it and generates code from it may be another distinct piece. By so partitioning the compiler software for different languages, a given code generation piece might be used with different AST generation pieces. Of course, the code generation piece must accept any AST that any of the AST generators can construct, which will increase its complexity.

We shall see that code generation and certain optimizations are facilitated by an AST representation.

Consider grammar G_0 :

$$\begin{aligned} E &\rightarrow E + T \\ E &\rightarrow T \\ T &\rightarrow T * F \\ T &\rightarrow F \\ F &\rightarrow (E) \\ F &\rightarrow a \end{aligned}$$

The derivation tree for even simple expressions is rather large. For example, the expression " $a*(a+a)$ " has the derivation tree shown in figure 7.6. This tree has 7 leaf nodes and 11 interior nodes, yet the expression itself contains only two operators and three operands. Why all the complexity?

Most of the complexity arises from the form of the grammar. Every derivation step is reflected in a tree node, yet many of the derivation steps are merely to provide a suitable precedence for the operators $+$ and $*$ and to cause operations inside parentheses to be performed before those outside.

We can reduce the derivation tree to an AST by first removing all the tree links associated with the single productions, i.e., $E \rightarrow T$, $T \rightarrow F$, and $F \rightarrow a$. They serve a purpose in the derivation process, but no useful role in the final structure.

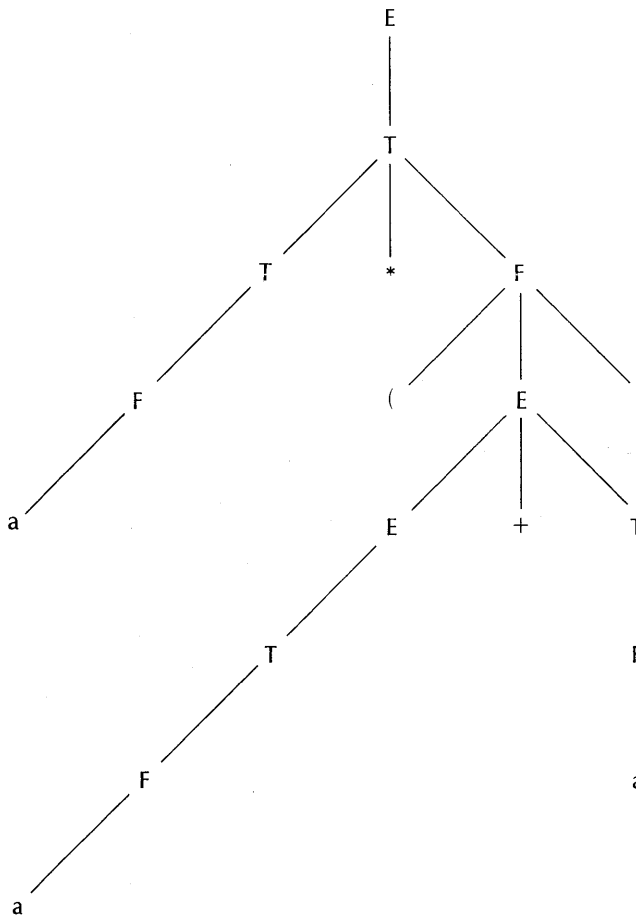


Figure 7.6. Derivation tree in G_0 for a simple expression $a*(a+a)$.

Next, the tree nodes containing the operators $*$ and $+$ can be reduced to a single node containing only the operator. We clearly no longer have a derivation tree, but so far have not lost any structural information either. At this point, our reduced tree looks like figure 7.7. It still says that the addition must be done first, then the multiplication. What about the parentheses? If we collapse the parenthesis production $F \rightarrow (E)$ will the operator ordering established by the parentheses be lost? The answer is no—the “a” node inside the parentheses came from a sequence of derivation steps that have already provided for the precedence of the $+$ operator inside the parentheses over the $*$ outside. We arrive at the AST in figure 7.8.

This tree is also the abstract-syntax tree for the expressions

$a*((a) + a)$
 $a*(a + (a))$
 $(a*(a + a))$

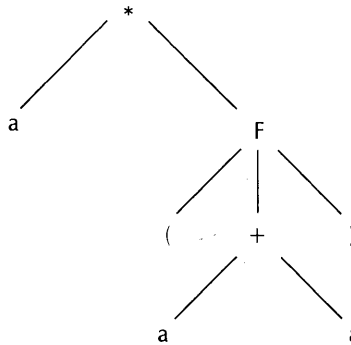


Figure 7.7. Derivation tree of figure 7.6 reduced by collapsing single productions and operator productions.

as should be obvious. Each of these is mathematically equivalent; we conclude that an abstract-syntax tree for an expression is a kind of canonical form for an equivalence class of expressions. However, the AST equivalence class is not complete, for it does not take into account commutativity or associativity of the operators $+$ and $*$. Consequently, the following expressions, although mathematically equivalent, yield different AST's:

$a*b + c$,
 $c + b*a$,
 $c + a*b$, etc.

We can easily generate an AST directly by some simple changes in the SDTS implementation. Instead of generating a complete translation tree, we simply generate an AST by pointer mechanisms similar to those in section 7.4. A grammar and the corresponding tree translation elements are illustrated in figure 7.9.

For a bottom-up parser, the rules in figure 7.9 can be understood as follows. Consider the $E \rightarrow E + T$ rule. When $E + T$ appears as the handle on the top of the parser stack, there will be two subtrees connected to the E and the T elements. (Nothing is connected to the “+”.) The translation rule

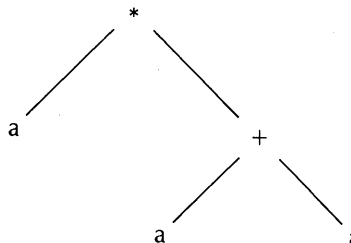


Figure 7.8. Abstract-syntax tree for expression $a*(a+a)$.

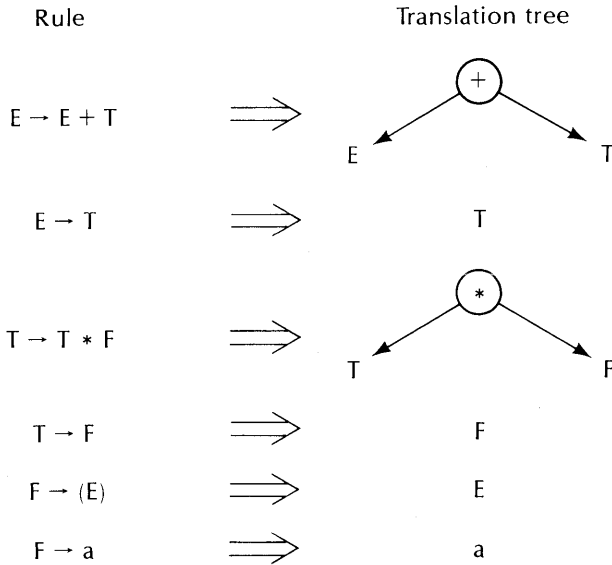


Figure 7.9. Direct generation of an abstract-syntax tree from grammar G_0 and its parser.

effectively says: “Create a new node, labeled ‘+’, and give it two sons—the E and T subtrees. We now have a new tree rooted in the ‘+’ node; attach it to the E that replaces E and T on the stack.”

The translation rule $E \rightarrow T$ effectively is applied as follows. When T is the handle, it is attached to some subtree; merely attach this subtree to the E that replaces T in the stack.

Finally, the translation rule $F \rightarrow (E)$ calls for attaching the E subtree to the element F on the stack, and the rule $F \rightarrow a$ calls for attaching the terminal token “a” as a tree consisting of one node to the element F in the stack.

When the “accept” state is reached, we will have the AST attached to the start token on the stack top.

The general form of an AST is a tree whose interior nodes are operators and whose leaves are simple variables or constants. The operators may be of any variety, unary, binary, or n-ary. For example, an array reference can be represented by the AST shown in figure 7.10, where X is a multidimensional variable, and $e_1, e_2, \dots, e_n$ represent expression AST’s for the indices; the reference in source form would look like this:

$$X(e_1, e_2, \dots, e_n)$$

A procedure call AST would be identical in form, except that the operator would be different, of course, and a different class of actual parameters might be permitted by the language.

An AST can represent control structures and declarations as well. For example, a CASE statement can be modeled as an AST as shown in figure

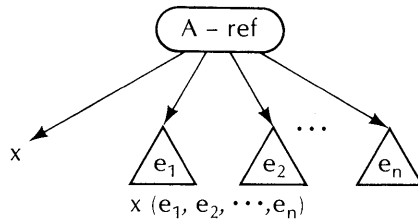
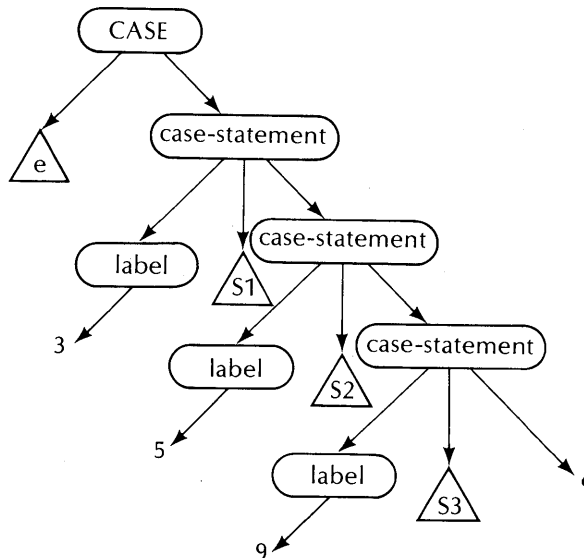


Figure 7.10. An array reference or procedure call abstract-syntax tree.

7.11. The nodes labeled “case statement” can all be the children of the CASE node, or can form a parent-child chain as shown, the last one with a nil right-most child.

Considerable freedom in the definition of an AST exists. However, an AST should satisfy several properties:

1. Each interior node must carry an operator with uniquely defined properties, e.g., number, kind, and ordering of children.



```

CASE e OF
  3: S1;
  5: S2;
  9: S3
END;

```

Figure 7.11. A CASE statement expressed as an abstract-syntax tree.

2. Every operator should carry some semantics and not just be a “place holder.” The test is whether it can be removed without losing information in some way.
3. An operator usually arises through some production with either an empty right member or more than one right member token. A single production almost invariably contributes nothing to the AST construction. This rule is not absolute, but is a reasonable guideline in designing an AST and linking it into a translation scheme.

Exercises

1. Write a set of productions for a case statement such that the component statements must carry one label each. Then devise an AST generator for your set.
2. Devise suitable data structures through which a table-driven, AST-generating, bottom-up SDTS can be designed. State the table-interpreting algorithm.

7.5.3. A Practical Bottom-Up Synthesis System

Figure 7.12 shows a set of three push-down stacks that can serve as the basis of a general purpose, bottom-up synthesis system. Each of the stacks grows downward, i.e., the “top-of-stack” is the bottom element.

Stack STATE is the usual LR(k) push-down stack. It contains state

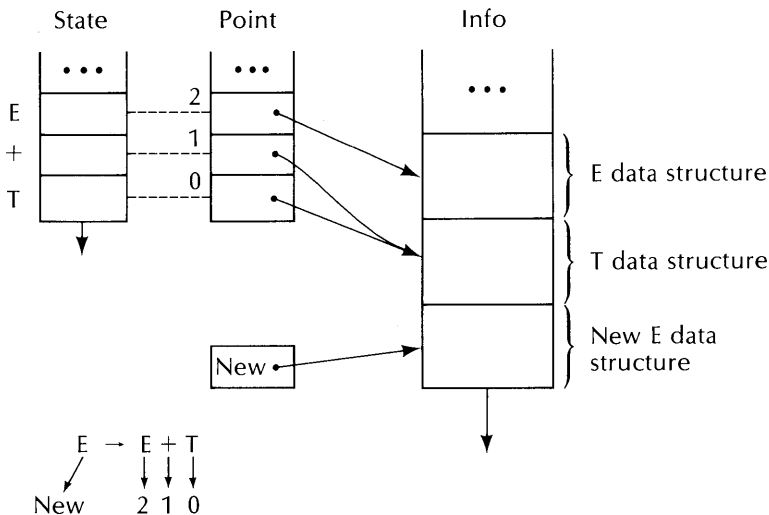


Figure 7.12. A generalized bottom-up translation system with stacks.

numbers used in the parsing process. On an APPLY step, the topmost set of state numbers corresponds to the handle. We show an APPLY of production $E \rightarrow E + T$ in figure 7.12.

Stack POINT contains pointers to an information stack INFO. Each element in POINT corresponds to an element in STATE. Most of the terminal elements and some of the nonterminal elements have an empty data structure in INFO, for example, the terminal token “+”, which points to the same element as the nonterminal “T”. Since some terminal elements carry information (e.g., identifiers), the POINT stack will correspond exactly to the STATE stack.

In addition to the three stacks, one cell for a pointer NEW is provided.

Figure 7.12 shows the state of the system just before an APPLY operation is executed. Symbols E and T correspond to two data structures in INFO, pointed to by POINT[2] and POINT[0], respectively. An empty area for a “new” E (corresponding to the left part of the rule $E \rightarrow E + T$) has been provided on the INFO stack top, and pointer NEW is assigned to it.

We may now perform some sort of operation on the old E and T data structures, either modifying them, or using them to construct a new data structure. There are three general end results of such operations, illustrated in the next three figures.

Figure 7.13 shows a PASS. The new data area and all but the deepest in the stack of the old areas are dropped. Two elements are dropped from the POINT and STATE stack in the example. Note that POINT[0] continues to point to the same structure as before. This operation is useful for single productions of the form $A \rightarrow B$, requiring no action. It can be seen that the data structure associated with B is simply “passed along” to A. Of course, the data carried by the E structure may be modified.

Figure 7.14 shows a KEEP. Here, we keep the old data structures and the new one (presumably filled with useful information by this time). However,

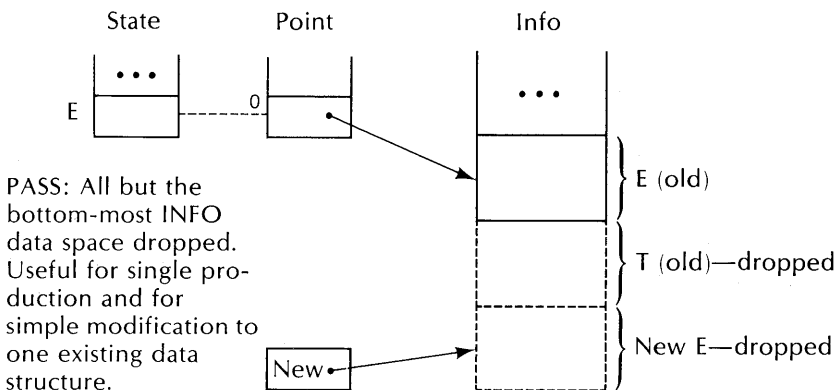


Figure 7.13. The PASS operation (see figure 7.12).

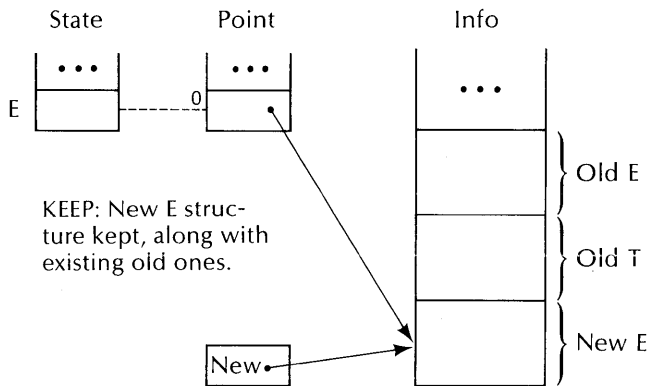
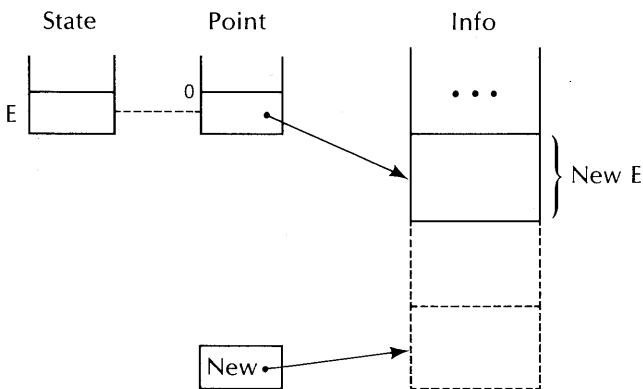


Figure 7.14. The KEEP operation (see figure 7.12).

the old pointers in POINT are lost and replaced by the NEW pointer to the new one. This operation is useful in building a tree structure on the stack. The new E structure could carry an operator (e.g., “+” in this example) and pointers to the old E and T structures. Note that we have an opportunity to rearrange the old E and T children of the new parent node E.

Figure 7.15 shows a REPLACE. Here, information in the old data structures is used to construct a new one, but then the old structures are discarded by “sliding” the new structure into their place. The old pointers in POINT are discarded and replaced by a pointer to the new structure. The net effect is the same as a KEEP, except that useless old information is removed from INFO, releasing space for new information.

The use of REPLACE implies that the replacement information must be



REPLACE: New E area kept, but old areas removed and stack reduced.

Figure 7.15. The REPLACE operation (see figure 7.12).

built in some temporary space while the existing space is being accessed; then the temporary space is moved into INFO.

Fortunately, REPLACE is unnecessary in many languages. Suppose that the grammar contains a nonterminal N such that N 's data structure is empty, and the data structure of every nonterminal X that can precede N in the stack is also empty. Then the INFO stack may simply be reset on any production $N \rightarrow w$, and no special attempt to reclaim space need ever be made. For example, in Fortran or Basic, the INFO stack may be cleared at the end of every logical line (a line with its continuations). In Algol, the INFO stack might be cleared at the end of every statement or declaration.

Of course, the STATE stack cannot be reset in Algol; it will contain essential information regarding the blocks.

Organization of a Synthesis System

We may now describe the organization of a synthesis system for a compiler as follows:

1. Design a data structure for each of the grammar's tokens, terminal and nonterminal. (Many of these will be empty.) Each structure will occupy a certain number of words or bytes in INFO. Some may occupy a variable number, however, the system can be more efficient if each structure has a fixed length.

For example, if the scanner recognizes each identifier as a unit (i.e., the grammar contains $\langle \text{identifier} \rangle$ as a terminal token), then the data structure for $\langle \text{identifier} \rangle$ might be:

```
type IDENT = record LENGTH: integer;
                    NAME: array[1..15] of char
end
```

As another example, suppose the grammar contained a production

$\langle \text{array-spec} \rangle ::= \text{array} [\langle \text{number} \rangle : \langle \text{number} \rangle] \text{ of } \dots$

(to simplify a Pascal structure), where the two $\langle \text{number} \rangle$ elements are also nonterminals returned by the scanner as numbers. Then a suitable data structure for $\langle \text{array-spec} \rangle$ might be:

```
type ARRAYSPEC = record LOWER: integer;
                       UPPER: integer
end
```

2. Upon an APPLY, the pointers in POINT will correspond to certain data structures in the INFO stack. Pascal provides no direct means of

associating a POINT pointer with a type, but we may get around that by statements of the following kind:

```
var S0: ↑EXPR;      {new pointer to an EXPR type}
S0:=POINT[0];      {pointer now given a value}
```

(The second statement requires a Pascal compiler extension.) Similarly, pointer NEW must be associated with the data structure of the left part of the production rule. We could devise a system that performs such pointer assignments automatically, or simply do it as needed as part of each apply action.

3. We next design a synthesis operation for each production. The nature of the operation is usually clear from the data structures involved, and what we wish to accomplish at the moment at which that particular production is involved in the bottom-up parse.

For example, each of the productions $E \rightarrow E + T$, $T \rightarrow T * F$ will call for adding a node to an AST being built in INFO. A production of the form

$$\langle \text{statement} \rangle ::= \langle \text{identifier} \rangle := \langle \text{expr} \rangle$$

will call for evaluation (through code emission or interpretation) of the $\langle \text{expr} \rangle$ AST, and then emitting a “STOR $\langle \text{identifier} \rangle$ ” instruction.

4. At the end of the synthesis operation, one of the three operations KEEP, REPLACE, or PASS is selected.

The parser will manage the STATE and POINT stacks. A parser SHIFT step may require the introduction of a terminal token data structure into INFO; this structure must therefore be supplied by the scanner and written into INFO.

Example. Let us build an AST with this system for the grammar G_1 given below:

1. $E \rightarrow E + T$
2. $E \rightarrow E - T$
3. $E \rightarrow - T$ {unary minus}
4. $E \rightarrow T$
5. $T \rightarrow T * F$
6. $T \rightarrow T / F$
7. $T \rightarrow F$
8. $F \rightarrow (E)$
9. $F \rightarrow \langle \text{ident} \rangle$

10. $F \rightarrow \langle \text{ident} \rangle (E)$ {an indexed reference}
11. $F \rightarrow \langle \text{const} \rangle$ {a decimal number}

Grammar G_1 is clearly an extension of G_0 to include subtraction, division, unary minus, identifiers, and indexed identifier references.

We need only one data structure, an EXPR type with several cases as follows:

```

type EXPR = record case CODE: set of (binop, unop, var,
                                     inxvar, const) of
  binop: (OPERATOR: set of (add, sub, mlt, div);
         LEFT: ↑EXPR;
         RIGHT: ↑EXPR);
  unop: (CHILD: ↑EXPR); {only one unary operator: -}
  var: (SYMPNTR: ↑SYMTAB); {pointer to token table}
  inxvar: (SYMPNTR: ↑SYMTAB;
          INDEX: ↑EXPR);
  const: (CONSTVAL: integer)
end

```

The apply operations for each production may now be written as follows. We have written out specific assignments to the pointers S1, S2, etc., corresponding to the POINT stack contents. This could have been done automatically before each apply operation.

Production 1: $E \rightarrow E + T$

```

with NEW do
begin {NEW assumed set to top of INFO stack}
  CODE:=binop;
  OPERATOR:=add;
  LEFT:=POINT[2];
  RIGHT:=POINT[0]; {pointers to the left and right
                   expressions}
end;
KEEP; {this builds upon existing structures}

```

Production 2: $E \rightarrow E - T$

Same as production 1, except “sub” instead of “add.”

Production 3: $E \rightarrow - T$

```

with NEW do
begin
  CODE:=unop;
  CHILD:=POINT[0];  {pointer to T part}
end;
KEEP;

```

Production 4: $E \rightarrow T$

PASS; {amounts to a do-nothing}

Production 5: $T \rightarrow T * F$

Same as production 1, except “mlt” instead of “add.”

Production 6: $T \rightarrow T / F$

Same as production 1, except “div” instead of “add.”

Production 7: $T \rightarrow F$

PASS;

Production 8: $T \rightarrow (E)$

```

with NEW do
begin
  POINT[2]:=POINT[1];

```

Production 9: $F \rightarrow <\text{ident}>$

```

with NEW do
begin
  CODE:=var;
  SYMPNTR:=FETCH(POINT[0]);
REPLACE; {essentially a PASS, except that we want a
          pointer to the E, not to the left parenthesis}
end;

```

{FETCH is a token table function. It locates the identifier pointed to by POINT[0] and returns a pointer to its attributes. Hence we have effectively attached the attributes of the identifier to the data structure being built. Failure to find the identifier is an error condition that we shall not deal with here.}

end;

```
REPLACE;    {we don't need the identifier name anymore}
```

Production 10: $F \rightarrow \langle \text{ident} \rangle (E)$

with **NEW** do

begin

CODE:=inxvar;

SYMPNTR := FETCH(POINT[3]); {the <ident> has index 3}

```
INDEX := POINT[1];
```

end;

```
KEEP;      {keep the new one and previous ones}
```

Production 11: $F \rightarrow \langle \text{const} \rangle$

```
type C=CONSTANT: integer;
```

```
var P : ↑C;    {pointer to type C}
```

with **NEW** do

begin

CODE:=const;

P:=POINT[0];

```
CONSTVAL:=P↑.CONSTANT;  {fetch the constant value
                           from INFO}  end;
```

REPLACE; {don't need the old constant anymore}

Figure 7.16 shows the structure of INFO upon parsing the expression $I1(X-15)+IM2$. POINT[0] points to a binary ADD node, with left child $I1(X-15)$ and right child IM2. The IM2 node is a “variable” node, pointing to the attributes of IM2 in the token table. The left child node $I1(X-15)$ is an “inxvar” node, etc. We clearly have an AST, and the identifier attribute association has been made.

Evaluation of an AST

Now suppose that the AST is evaluated by some tree-walking or transforming automaton. The evaluation yields a block of output code, and

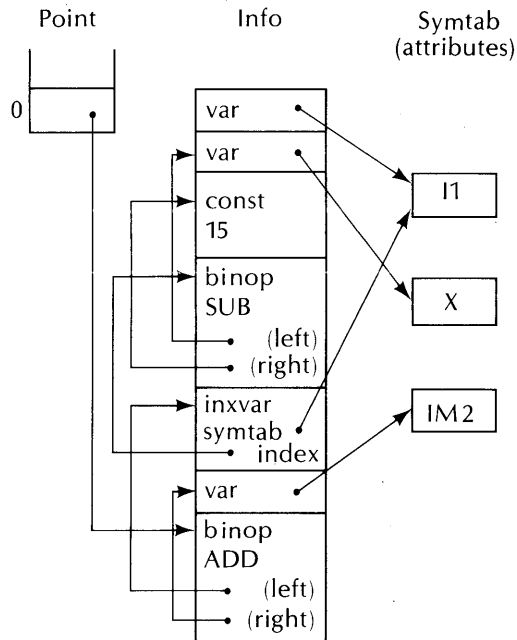


Figure 7.16. Appearance of the generalized translation system (see figure 7.12) after translation of the expression $I1(X-15)+IM2$.

we then no longer need the tree. The evaluation and removal of the tree is triggered by some other production containing E in its right part, demanding evaluation, e.g.,

$$S \rightarrow V := E$$

a replacement statement. Of course, the “ V ” is some variable and corresponds to some variable type such as

```

type VARTYPE:
  record case: set of {simple, inx} of
    simple: (SYMPNTR: ↑SYMTAB);
    inx: (SYMPNTR: ↑SYMTAB;
         INDEX: ↑EXPR)
  end

```

We let the “ S ” data structure be empty, so that upon completing this production, the INFO stack will be emptied as well. The following program illustrates a recursive top-down AST evaluator. It generates stack code. The target machine has a separate index register whose value is first developed on the stack, then placed in the index register by STAX. The instruction LA is a “load address”; it determines the address of the following operand and pushes

it. TOS refers to the stack top; TOS - 1 to the operand just below the stack top operand.

```

var S2=↑VARTYPE; {pointer to VARTYPE}
S2:=POINT[2];

if S2↑.CODE = inx then
begin {left member is indexed variable}
  EVALUATE(S2↑.INDEX); {get index value on stack}
  EMIT('STAX;'); {code to push into X register}
  EMIT('LA ');
  EMIT(S2↑.SYMPNTR.ADDR); {address connected to vari-
                           able}
  EMIT(';X;'); {indexed, and closing semicolon}
end
else
begin {simple variable}
  EMIT('LA ');
  EMIT(S2↑.SYMPNTR.ADDR); {address connected to vari-
                           able}
  EMIT(';'); {the closing semicolon}
end;

EVALUATE(POINT[0]); {evaluate right member expression}
EMIT('STOR TOS-1,I;'); {do the store through stack label}
EMIT('DEL;'); {drop the label left on stack}
REPLACE; {empty replaces all the stuff used in these
          productions}

```

Exercises

1. Develop a set of productions, a set of data structures and apply actions to generate an AST of the form shown in figure 7.10, for an array reference or procedure call. Note that the "A-ref" node has an indefinite number of children. It must therefore be written on the INFO stack after each of the expansion trees e1, e2, ... are written. *Hint*: write right-recursive productions for the expression list, and examine the sequence of bottom-up apply actions for such productions.
2. Develop productions and data structures for a CASE statement whose AST has the form shown in figure 7.11.
3. Consider the AST evaluator given at the end of section 7.5.3.

- (a) Give the AST and the emitted assemble code for each of the following replacement statements:

$$A := B + 3*(C - D);$$

$$X[I - J] := Y[K] - 15;$$

- (b) Note that the address of the left-hand member of a replacement statement is pushed in the run-time stack before the right-hand member is evaluated. Suppose instead the index of the left-hand member could be evaluated after the right-hand member is evaluated. How could this be done?
- (c) Instead of pushing the address of R in a nonindexed replacement statement $R := E$, it would be more efficient to simply emit "STOR R" after evaluating E. Show how to modify the evaluator program to do this.
4. Develop a system that builds a linear linked list of INFO elements, given (a) right-recursive productions and (b) left-recursive productions. What is the order of the linked list relative to the order of the source list in each case?

7.6. Bibliographic Notes

The notion of an SDTS was first formalized by Lewis and Stearns (Lewis [1968]). The top-down and bottom-up transducers are adapted from Aho and Ullman (Aho [1972a]), chapter 3. A general discussion of tree generation and tree transformations may be found in DeRemer [1974a].

A great deal of literature exists on automatic translator writing systems. An old, but comprehensive, review is in Feldman and Gries' paper (Feldman [1968]). A more recent survey is given by Griffiths [1974c]. Specific syntax-directed translators are described in Koster [1974], Brooker and Morris (Brooker [1963]), and Metcalfe [1964].

STATIC REPRESENTATIONS OF DATA OBJECTS

Every common programming language permits a programmer to invent names for the entities that he wishes to have the computer manipulate when his program is executed. Properties may be assigned to his names through special language forms called *declarations*. These properties can then be used to define a particular class of operations for the named entities. For example, an addition operator “+” can be used for a variety of different kinds of addition—real, vector, fixed point—by assigning a type to its operands through declarations. In this way, a few operators can be used for a large number of related mathematical operations, increasing the power and clarity of the language.

Some languages permit rather arbitrary aggregates of data to be associated with a single name, for example, a matrix, a tree, or a linked list. The elements of the aggregate can be accessed through various devices, such as indexing or the use of composite names.

This chapter deals with the static association of user-defined names with attributes. By *static*, we mean those associations that the compiler must keep track of, as opposed to those that must exist when the compiled program is executed. There are several aspects to static name association:

- Names will appear in the source more or less at random. Their association with a set of attributes requires some means of efficiently locating a particular name in a table.
- Several languages provide static scoping of user names through blocks. A name will be useful only within its block. The same name may appear in different blocks, but will represent different entities.
- A compiler may operate through one or several passes through the source or through an abstract syntax tree generated on the first pass. In general, the attributes of the user names are needed in all the passes.
- If the source code is free of errors, the names are no longer needed once their attributes have been established and linked to the AST. However, the compiler must be prepared to deal with source errors, and error messages are often enhanced by including appropriate names.

8.1. Symbols, Declarations and Uses

Different kinds of symbols appear in most source languages. There are symbols such as $+$, $*$, $($, $)$ that carry at most one or two specific, fixed meanings in the language. There are strings of characters that represent constants, such as quoted strings and numbers, called *literals*. Finally, there is a class of user-defined names, called *identifiers* that carry no inherent meaning, but rather are assigned meaning through their context within the source program.

An identifier is typically a string starting with a letter and containing only letters and digits. It may have a bounded length (one to seven characters in Fortran, for example), or unbounded. The strings in the source that comprise identifiers are usually distinguished by a lexical analyzer. As we have seen in chapter 3, the task of distinguishing identifiers from reserved words in certain languages is not easy.

An identifier can be associated with any of a number of entities in a programming language, for example:

- As a reference to some data area. The area may contain a simple fixed datum or several data associated in some fashion.
- As a component of a *composite name*. The composite name may refer to a data area, but the components refer only to some subset of the data area. Composite names consist of a sequence of identifiers separated by some special separator symbol, e.g. COMPANY_PAYROLL in Cobol.
- As a reference to a statement location in a program, i.e., a *statement label*. We shall use the term *label* in this sense hereafter.
- As a procedure name.
- As a macro name.
- As a procedure or macro parameter.
- As a file, or a program, or a device connected to an input-output port of the computer.

8.1.1. Attributes, References and Declarations

The set of meanings associated with an identifier is called its *attributes*. Some examples of attributes are:

- Whether the identifier represents data, a procedure, a statement label, a file, a macro, etc.
- If it represents a datum, which of several possible kinds of values it may take on, its location, and its links to related data.

- If it represents a procedure, the location of the procedure, the number and kinds of its parameters, whether it is user or system defined, whether it can be called by a user or not, etc.
- If it represents a file, the characteristics of the file—record size, whether fixed or variable length, whether sequential, random access, etc.

Clearly, attributes require some kind of classification scheme, as each of the different kinds of identifiers in the language require a different form of attribute specification. The number of attributes may also depend on more than one declaration.

The compiler data structure that associates identifiers with their attributes is called a *symbol table*. Often the identifiers are carried in a separate table called a *name table*, with the attributes in a separate table called an *attribute table*. We shall see that in most cases, the symbol table and its associated abstract syntax tree (AST) can be carried in a single push-down stack.

A statement whose principal purpose is to assign attributes to some identifier is called a *declaration*. An identifier is said to be *declared* when it has appeared in a declaration. An identifier is said to be *referenced* or *used* in a statement in which it appears, but in which no attributes are added to the identifier's attribute set. Sometimes, an appearance of an identifier is both a declaration and a use. For example, the Fortran statement

$$\text{SAM} = \text{I} + 1$$

in which I appears for the first time in the source, is both a declaration and a use of I. Under the Fortran rules, identifiers beginning with I, J, K, L, M, or N are assumed to be simple integer variables, unless there is a preceding declaration to the contrary.

In general, a declaration usually causes no object code to be emitted, except possibly to allocate space for data. A reference is usually associated with some generated object code. Of course, a macro name reference results in the generation of an expansion source string and may not be associated with any object code generation.

An identifier that only appears in declarations and is never referenced is useless—it need never appear at all. An identifier that appears in two or more conflicting declarations in the same static scope is said to be *multiply declared*. An identifier that appears in a reference, but never in a declaration is said to be *undeclared*. Whether undeclared identifiers are permitted depends on the language—Fortran permits a class of undeclared identifiers, while Pascal does not.

All the declarations of some identifier must usually precede any reference of the identifier. There is no particular implementation reason for this language policy, as a multi-pass compiler can deal with declarations and references in any order. However, it is good programming style to group the declarations together in a section of the source program that precedes all the

executable statements. When declarations must precede references, the compiler can generate completed object code in one pass, except for forward branches.

8.2. Scope of Identifiers

Every identifier possesses a region of validity within the source program, called its *static scope* of definition. An identifier is *available* within its scope and *unavailable* outside its scope. A reference to some datum through its identifier is valid only if the reference lies within the identifier's scope.

The languages Fortran and Basic have very simple scope rules—the scope of any identifier is an entire program or procedure, and a procedure cannot be nested within another procedure.

In a block-structured language, the scope of an identifier is that of the block in which it is declared. However, the same identifier can be declared several times within the same block without conflict. The scope rules may be expressed as:

1. A *block* is a sequence of source with the structure HEAD DECLS STMTS TAIL where HEAD may be the keyword BEGIN or a procedure declaration head, DECLS are some declarations, STMTS are executable statements, and TAIL is usually the keyword END.

2. Two blocks A and B must be disjoint, or one must be contained in the other. If block A is contained in B, A must be among the DECLS or STMTS of B. Block A is then said to be *nested* in block B, and block B is said to *cover* block A.

3. An identifier declared in DECLS in a block A is available throughout block A, with the exception noted in (4) below. It is not available outside block A.

4. An identifier may be declared in a block A and also in a block B nested in A. The same identifier then represents two different objects. They are best distinguished by renaming the B object for each appearance of the identifier in block B. In any case, the A object is unavailable within block B.

5. The same identifier may be declared in two disjoint blocks A and B, and represent different objects. As in (4) above, we may rename the B object to eliminate confusion. Clearly, the A object is available only in block A and the B object only in block B.

These scope rules are of considerable value to programmers. A new block of source may simply be inserted into an existing program. If new temporary variables are needed in the new block, they may be declared within the block without any concern that one of their names may have already been used in a covering or disjoint block.

8.2.1. Single and Multiple Pass Compilers

A compiler is said to be *single-pass* if the object code can be fully generated in one scan of the source. It is said to be *multi-pass* if two or more scans of the source, or a source representation, are needed.

A single pass compiler almost always requires one or more tables or *fixup lists* to achieve its translation. A common problem encountered in a single pass compiler is the resolution of branches. A forward branch instruction must be emitted to the object code list before its target location is known. When the target location is found, the instructions must then be adjusted. Sometimes the adjustment is done by a *loader*, just before the program is executed; the loader then provides some of the services of a two pass compiler. Otherwise, the compiler must carry a list of instructions that are to be adjusted or “fixed up” when certain statements are located.

A multi-pass compiler can build a symbol table and an AST on the first pass, then scan through the AST one or more times to resolve any code or data locations before emitting object code. Some multi-pass compilers actually scan the source a second time, performing all the lexical analysis and symbol table lookups on each pass.

The principal advantage of multiple passes is that each pass can concentrate on just a few aspects of the compilation process. Each pass requires a fairly small program, but one that is substantially different from those of the other passes. A single pass compiler requires only one scan of the source, but must invoke all its translation algorithms and code fixups repeatedly throughout the single source scan.

The combination of multiple passes and block structuring requires a fairly elaborate symbol table structure. We shall examine three general cases—the “single scope” case which can serve for single or multiple passes, the “single pass, multiple scope” case, and the “multiple pass, multiple scope” case.

Single Scope Symbol Table

The symbol table and AST for a language with a single scope of variable names can be carried in one push-down stack, as indicated in figure 8.1. The attributes and names can be carried in a single data element. The same identifier cannot be declared twice, so that names and attributes carry a one-to-one association.

A name declaration requires a search for the name in the symbol table; if already present, we have a multiply declared name—an error or not, depending on the language. A reference also calls for a search to determine its attributes. The AST can be built with pointers as needed to the identifier attributes. Names are entered before any AST references can be built.

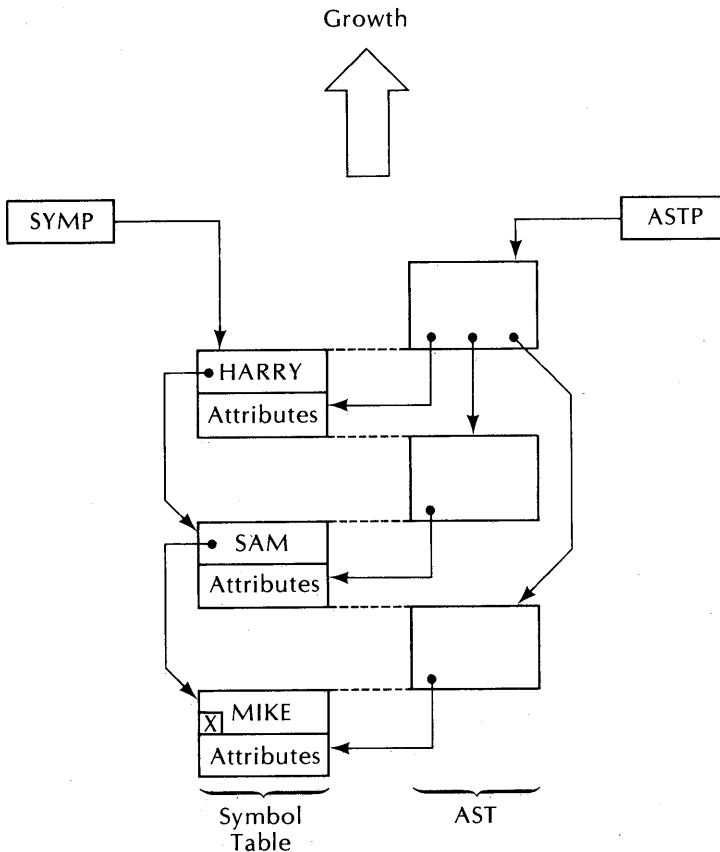


Figure 8.1. A single scope symbol table and abstract-syntax tree (AST). The AST and symbol table can be part of one push-down stack.

Single-pass, Multiple Scope Symbol Table

Again, the names, their attributes, and the AST can share one push-down stack. With multiple scopes, the same identifier can be associated with several different sets of attributes. Some identifiers such as I or N tend to appear again and again in a program. We can therefore achieve some economy of stack space by writing a given identifier in the stack only once, and then referring to it by pointers in the attribute entries. Figure 8.2 shows two scope levels with six data objects and four identifiers currently active. The corresponding program might be the following:


```

BEGIN
  var SAM, FRED, MARY: integer;
  .
  .
  .
  BEGIN
    var FRED, SAM, BILL: integer;
    {Symbol table illustrated for this scope}
    .
    .
    .
  END;
END;

```

The rules for constructing and maintaining this symbol table-AST structure are as follows:

Scope entry. Upon entering a new scope (a BEGIN or PROCEDURE), the stack top is marked (the horizontal dashed line in figure 8.2). This provides demarcations to separate the scopes in the stack.

Scope exit. Upon leaving a scope (through an END), the stack is popped to the top-most mark. The pointers SYMP and ASTP, to the top symbol and the AST root, respectively, are adjusted by following the element linkages down into the stack to just below the mark.

Declaration. Upon a new identifier declaration, a search is made for the name through the attribute pointer chain, starting at the SYMP attribute (stack top). The search should continue to the bottom of the stack, in order to share identifiers, however, the identifier is considered to be multiply defined only if it is found connected to an attribute above the topmost mark. If the identifier is not found, it must be added to the stack. In any case, a new attribute element is also added to the top of the stack.

Reference. Upon a reference of an existing identifier, a search is made for the name through the attribute elements, starting with SYMP. The search ends on the first matching identifier, although other attribute elements deeper in the stack may also point to the same identifier (they must be in covering scopes.) A reference usually results in some addition to the AST; the SYMP pointer is clearly available for this purpose.

At the end of a block, all the attributes, identifiers, and the AST portion associated with the block may be discarded. Of course, this is not done until the AST has been scanned, the block's code emitted, a symbol table map printed, etc.

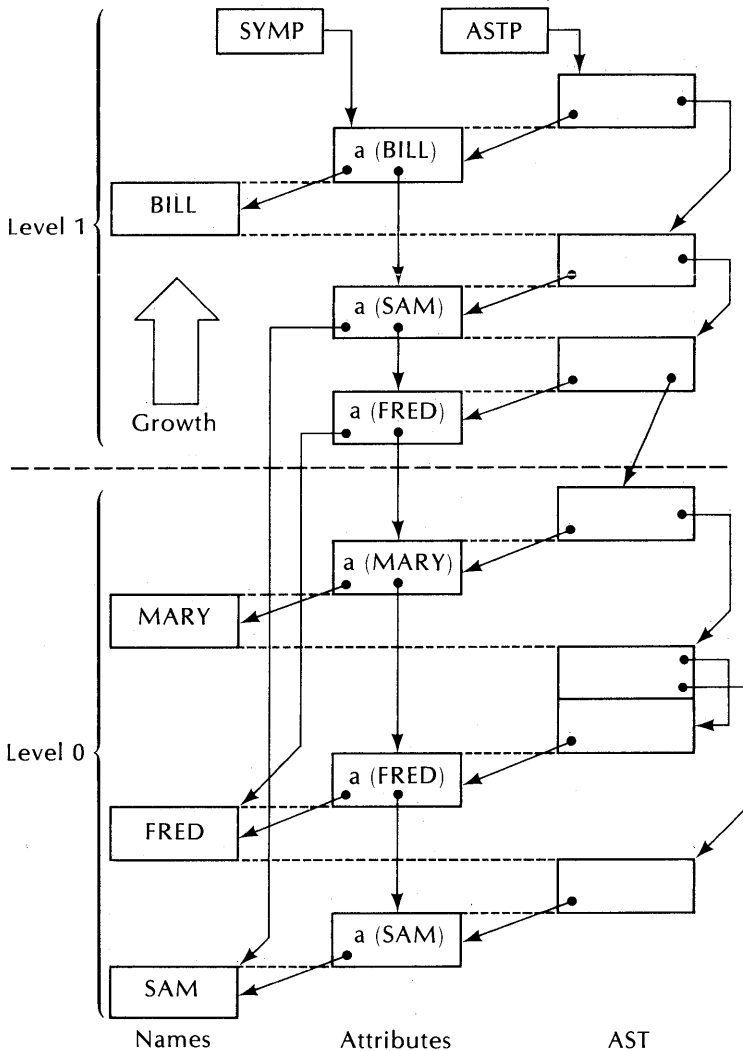


Figure 8.2. A single-pass multiple scope symbol table and AST, with name-sharing. All three structures can share one stack.

Multiple Pass, Multiple Scope Symbol Table

A multiple pass, multiple scope symbol table takes the form shown in figure 8.3. A common name table, with each name appearing exactly once can be used as in figure 8.2. However, the attribute fields must be organized as a tree, and carried through all the passes. The name table, attribute tree, and

AST are built in the first pass. Nothing is discarded at a block end, although some names and attributes might be written to secondary storage to conserve primary storage space.

On the second and subsequent passes, the AST is scanned, and its links to the attributes are used. The name table is unimportant, except as a resource in case program errors are found.

The tree of figure 8.3 corresponds to the following program:

```

BEGIN
  var a1, a2;
  BEGIN
    var a3, a4;
    BEGIN
      var a5, a6, a7;
    END;
    BEGIN
      var a8, a9;
    END
  END;
  BEGIN
    var a10, a11;
    BEGIN
      var a12, a13, a14;
      {SYMP pointer effective here}
    END
  END
END;

```

The attribute tree, name table, and AST may be part of one push-down stack; however, if we choose to discard the name table at the end of the first pass, it must be in a separate stack. A simple mark in the stack is inadequate as a scope delimiter. We need a mark position in each attribute element. In general, SYMP will be NIL or will point to some attribute element. The chain from SYMP down to the stack bottom will then represent a currently active set of identifiers from the innermost nested block to the outermost block. SYMP will move around the tree structure as we move from block to block in the source.

The rules for constructing the attribute tree on the first pass are as follows:

Block entry. Mark the attribute node pointed to by SYMP, then prepare to create a new son of that node.

Block exit. Move pointer SYMP down through the attributes associated with the current level until a marked attribute is found. *Note*—pointer SYMP may already point to a marked node.

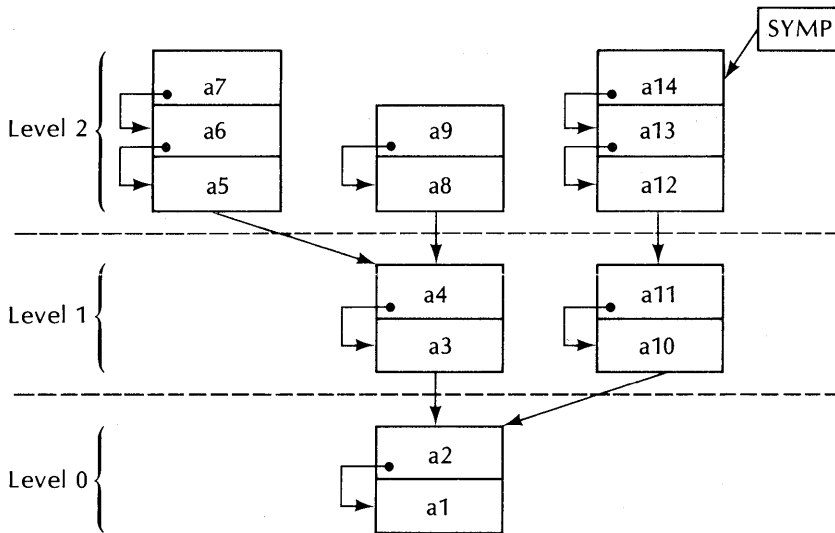


Figure 8.3. A multiple-pass, multiple scope symbol table. The attributes form a tree, with pointers to each other and to a name table. The AST points to the attributes. The structure can be built in one stack.

Declaration. A declaration calls for a new attribute element, to be linked to that pointed to by SYMP. The identifier search is along the tree path from SYMP to the stack bottom, following the attribute chains into the stack. Each attribute element points to some name in a name table (not shown in figure 8.3). A name is multiply declared if found in the current level, but not otherwise. The search should continue to the bottom of the stack if names are to be shared.

Reference. A reference calls for a search along the path from SYMP to the bottom of the stack. The identifier is undeclared if not found somewhere in the tree path.

On the second and subsequent passes, the AST is simply scanned through the links established in the first pass. SYMP is unnecessary.

8.2.2. Algol Statement Labels

In Algol 60, a label is considered declared when it appears as a statement label. Thus a use of a label can precede its declaration. Every other entity must be declared prior to its first use. For example, the following is a legal block in Algol 60:

```
begin
  go to LBL;      {use of LBL}
  I:=I+15;
  LBL: I:=I+1;    {declaration of LBL}
end;
```

The usual scope rules apply to labels—the scope of a label is the innermost block that contains the declaration. Thus in the above program segment, LBL is unavailable outside the begin-end pair.

In order to handle Algol labels, we assign one of the following three attributes to each statement label:

- D: declared only (no reference)
- RU: referenced at least once, but undeclared
- R: declared, and referenced at least once

State D applies to a label that has appeared as a statement label, but not yet in a GO TO. State RU applies to a label that has appeared in a GO TO, but not as a statement label. State R applies to a label that has appeared both ways. These three states permit the compiler to establish, at the end of a block, that:

- A label has appeared exactly once as a statement label
- A label has appeared at least once in a GOTO

The first condition is essential—if it does not hold, an error exists, and should be reported. A label appearing in the symbol table but not as a statement label implies that it has appeared in a GO TO, and that GO TO has no branch target. A label appearing more than once as a statement label is multiply defined.

If the second condition does not hold, a label has appeared only as a statement label, not in a GO TO, and may therefore be removed from the source. Failure of the second condition should be brought to the attention of the programmer, as he apparently had something in mind when he labeled the statement.

We can then implement the label states as follows:

Block entry. Mark the attribute stack as usual.

Block exit. Scan the current stack scope for the attributes RU and D. A “D” attribute can be considered a warning, but the “RU” attribute is an error. (However, there is a complication as we shall see in the next section.)

Statement label. If the label is not in the symbol table, then enter it with attribute D. If it is in the symbol table, it must be a label with attribute RU, R, or D. If R or D, we have a multiply declared label—an error. If RU, this appearance is legal, and the attribute must be changed to R.

Reference. Here, the label has appeared in a GO TO. If the label is not in the symbol table, then enter it with attribute RU. If it is in the symbol table, it must be a label with attribute RU, R, or D. Any of these is legal, however, attribute D is changed to R, while attributes RU and R are unchanged.

These rules apply only on the first pass. On subsequent passes, every label will be marked “R” in an error-free program. Its associated location will be

known only in terms of some node position in the AST. Eventually, the AST will be reduced to minimal form and code will be emitted. Only then will the absolute object code locations of the statement labels be known.

A Complication with Algol Statement Labels

Under the Algol scope rules, a GO TO in some block can refer to a statement label anywhere within a covering block, so that a transfer from within some nesting level can occur to outside the block.

Now consider the following program:

```

{1} begin {start of block B1}
    {2}   begin {start of block B2}
        {3}   go to L1;
            .
            .
            .
    {4}   end; {end of block B2}
    {5}   L1: . . .
    {6}   end; {end of block B1}

```

Upon encountering statement 3, containing the first appearance of the statement label L1, L1 must go into the symbol table. In the absence (so far) of any other information, it would be placed in the block B2 scope. Now at the end of block B2 (statement 4), the declaration of L1 has not yet appeared. Under Algol scope rules, the label need not be in block B2—it can be in some covering block. This causes a certain problem for the compiler upon reaching statement 4. An extension to the block exit mechanism is needed to deal with this situation.

Clearly, the needed extension is some means of moving the L1 attribute from the block B2 scope into the block B1 scope. This may have to be repeated at the end of block B1, if the label L1 has not yet been declared, until the end of the program is reached, if necessary. Only then may the compiler conclude that label L1 is undeclared. In this way, label L1 is “handed down” through the covering blocks until its declaring block is found, or until the end of the program is reached.

Moving an attribute from one scope into another is fairly easy in a single-pass compiler. In figure 8.2, when the end of a block is reached, that block’s attributes are discarded. Instead of discarding the undeclared labels, they may simply be kept in the stack, and the stack mark moved to cover them.

In a multi-pass compiler, attribute movement can only be achieved by organizing all the attributes as linked lists. In figure 8.3, the attributes of each block are shown as contiguous, but they must in fact be linked by pointers.